ktunotes.in

GET IT ON Google Play



KTU NOTES

The learning companion.

KTU STUDY MATERIALS | SYLLABUS | LIVE NOTIFICATIONS | SOLVED QUESTION PAPERS

Website: www.ktunotes.in

CST414
DEEP LEARNING
MODULE 1-PART 1

Ktunotes.in

SYLLABUS

Module-1 (Neural Networks)

Introduction to neural networks -Single layer perceptrons, Multi Layer Perceptrons (MLPs), Representation Power of MLPs, Activation functions - Sigmoid, Tanh, ReLU, Softmax. , Risk minimization, Loss function, Training MLPs with backpropagation, Practical issues in neural network training - The Problem of Overfitting, Vanishing and exploding gradient problems, Difficulties in convergence, Local and spurious Optima, Computational Challenges. Applications of neural networks.

KTUQbank.in

Text Books

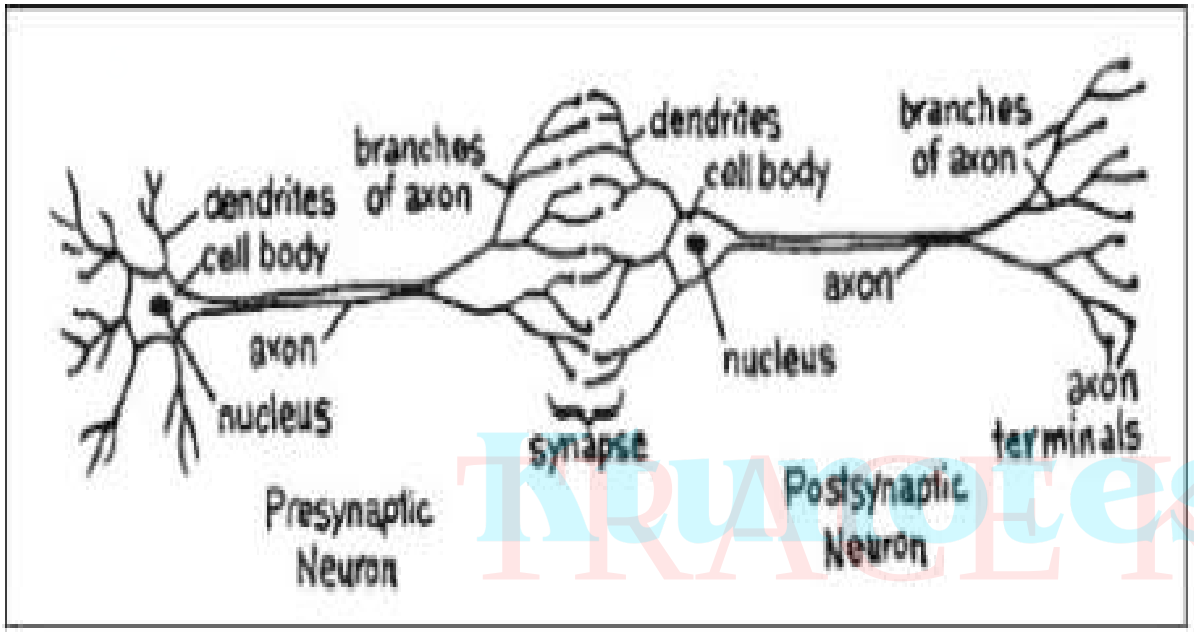
- 1. Goodfellow, I., Bengio,Y., and Courville, A., Deep Learning, MIT Press, 2016.
- 2. Neural Networks and Deep Learning, Aggarwal, Charu C.
- 3. Fundamentals of Deep Learning: Designing Next-Generation Machine Intelligence Algorithms (1st. ed.). Nikhil Buduma and Nicholas Locascio. 2017. O'Reilly Media, Inc

KNOWLEDGE KIT

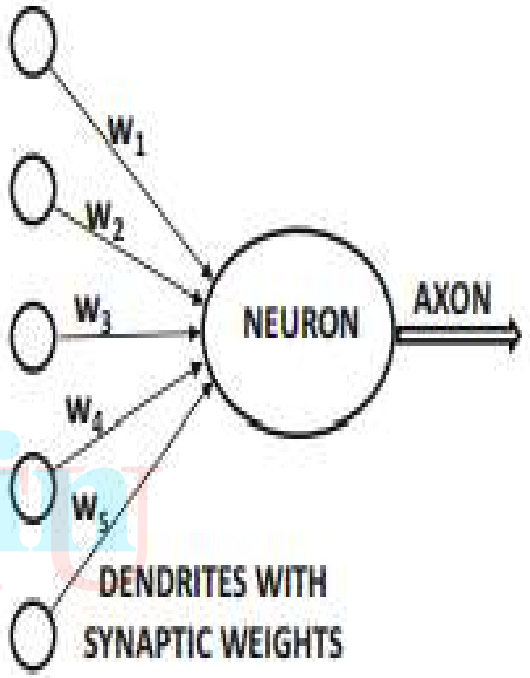
INTRODUCTION TO NEURAL NETWORKS

- Artificial neural networks are popular machine learning techniques that simulate the mechanism of learning in biological organisms.
- The human nervous system contains cells, which are referred to as **neurons**.
- The foundational unit of the human brain is the neuron. A tiny piece of the brain, about the size of grain of rice, contains over 10,000 neurons, each of which forms an average of 6,000 connections with other neuron
- The neurons are connected to one another with the use of **axons and dendrites**, and the connecting regions between axons and dendrites are referred to as synapses

Knoteskin



(a) Biological neural network



(b) Artificial neural network



The strengths of synaptic connections often change in response to external stimuli. **This change is how learning takes place** in living organisms.

- This biological mechanism is simulated in artificial neural networks, which contain **computation units that are referred to as neurons.**
- The computational units are **connected to one another through weights**, which serve the same role as the strengths of synaptic connections in biological organisms.
- After being weighted by the strength of their respective connections, the inputs are summed together in the cell body. This sum is then trans- formed into a new signal that's propagated along the cell's axon and sent off to other neurons.

KTUCLEKTU

An artificial neural network computes a function of the inputs by propagating the computed values from the input neurons to the output neuron(s) and using the weights as intermediate parameters.

- **Learning occurs by changing the weights** connecting the neurons. Just as external stimuli are needed for learning in biological organisms, the external stimulus in artificial neural networks is provided by the training data containing examples of input-output pairs of the function to be learned.
- For example, the training data might contain pixel representations of images (input) and their annotated labels (e.g., carrot, banana) as the output.

HRACLES.in

- The training data provides feedback to the correctness of the weights in the neural network depending on how well the predicted output (e.g., probability of carrot) for a particular input matches the annotated output label in the training data
- The weights between neurons are adjusted in a neural network in response to prediction errors.
- This ability to accurately compute functions of unseen inputs by training over a finite set of input-output pairs is referred to as **model generalization**.
- The primary usefulness of all machine learning models is gained from their ability to generalize their learning from seen training data to unseen examples.

KRACE.ac.in

- A key advantage of neural networks over traditional machine learning is that the former provides a **higher-level abstraction of expressing semantic insights about data domains by** architectural design choices in the computational graph.
- The second advantage is that neural networks provide a simple way **to adjust the complexity of a model by adding or removing neurons from the architecture according to the availability of training data** or computational power.

Krunotes.in

- In 1943 by Warren S. McCulloch and Walter H. Pitts.,
- Just as in biological neurons, our artificial neuron takes in some number of inputs, x1 , x2 , . . . , xn , each of which is multiplied by a specific weight, w1 ,w2 , . . . ,wn .
- These weighted inputs are, as before, summed together to produce the **logit** of the neuron,

$$z = \sum_{i=0}^{n} w_i x_i$$

- In many cases, the logit also includes a bias, which is a constant . The logit is then passed through a function f to produce the output y = f(z) . This output can be transmitted to other neurons

Kkuncotes.in

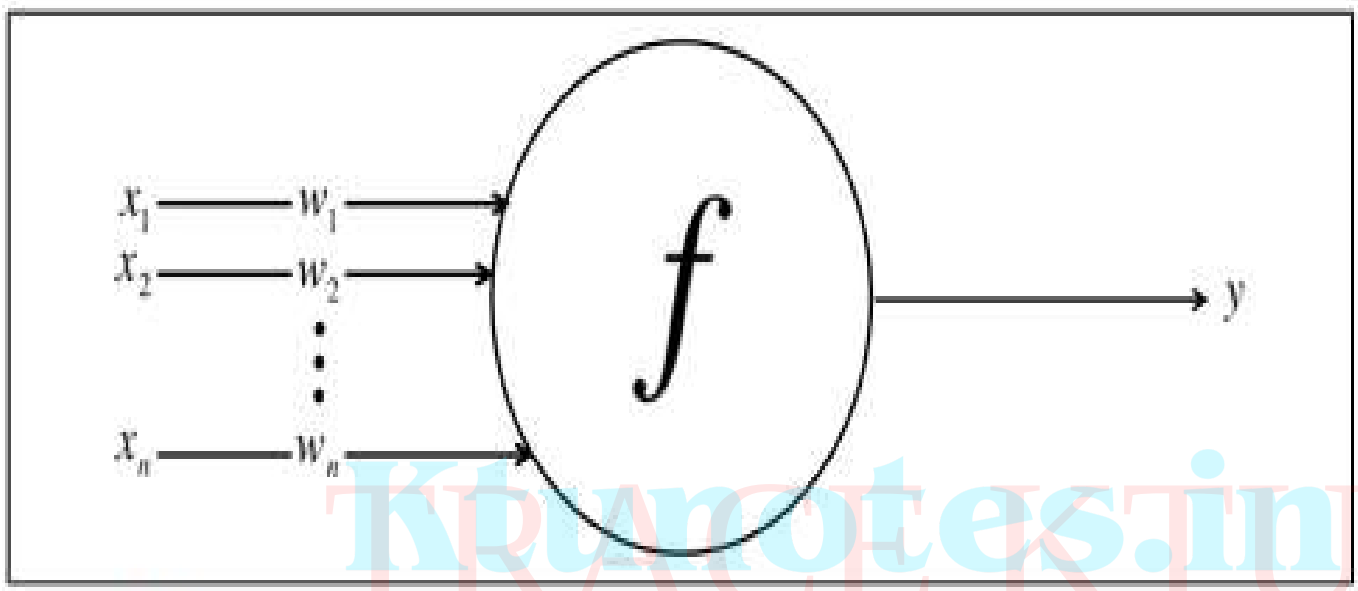


Figure 1-7. Schematic for a neuron in an artificial neural net

Krantes.in

Let's reformulate the inputs as a vector x = [x1 x2 … xn] and the weights of the neuron as w = [w1 w2 … wn]. Then we can re-express the output of the neuron as , $y = f(\mathbf{x} \cdot \mathbf{w} + b)$, where b is the bias term

A **Perceptron** is an **Artificial Neuron**

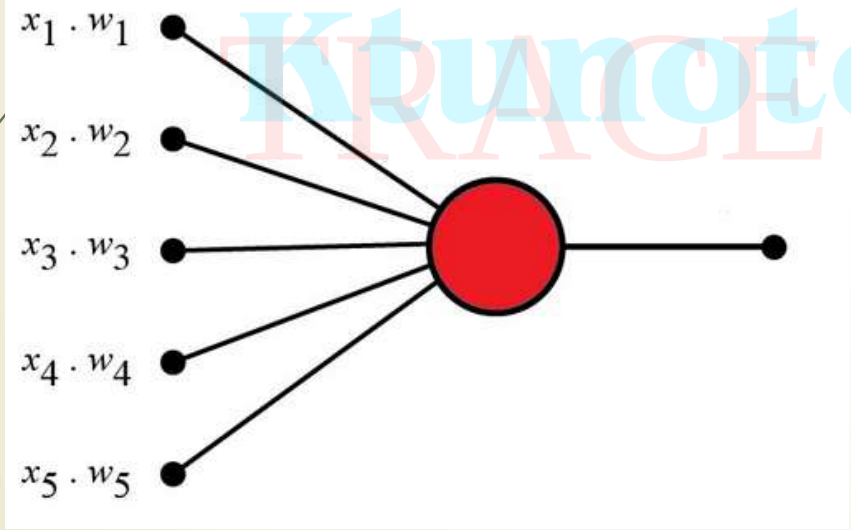
- It is the simplest possible **Neural Network**
- **Neural Networks** are the building blocks of **Machine Learning**.
- **Frank Rosenblatt**
- In **1957,** He "invented" a **Perceptron** program, on an IBM 704 computer at Cornell Aeronautical Laboratory.
- Scientists had discovered that brain cells (**Neurons**) receive input from our senses by electrical signals.
- The Neurons, use electrical signals to store information, and to make decisions based on previous input.
- Frank had the idea that **Perceptrons** could simulate brain principles, with the ability to learn and make decisions.

KRAnotes.in

- **The Perceptron**

  The original **Perceptron** was designed to take a number of **binary** inputs, and produce one **binary** output (0 or 1).

- The idea was to use different **weights** to represent the importance of each **input**, and that the sum of the values should be greater than a **threshold** value before making a decision like **true** or **false** (0 or 1).



Kraoles.in

- **The Perceptron Algorithm**
- Frank Rosenblatt suggested this algorithm:

1. Set a threshold value
2. Multiply all inputs with its weights
3. Sum all the results
4. Activate the output

- ❖ Eg:
- Imagine a perceptron (in your brain).
- The perceptron tries to decide if you should go to a concert.
- Is the artist good? Is the weather good?
- What weights should these facts have?

KRACE KTU

Criteria	Input	Weight
Artists is Good	**x1** = 0 or 1	**w1** = 0.7
Weather is Good	**x2** = 0 or 1	**w2** = 0.6
Friend will Come	**x3** = 0 or 1	**w3** = 0.5
Food is Served	**x4** = 0 or 1	**w4** = 0.4
Alcohol is Served	**x5** = 0 or 1	**w5** = 0.4

Kvanotes.in

inputs(x1,x2,x3,x4,x5) = [1, 0, 1, 0, 1]

Weights(w1,w2,w3,w4,w5) = [0.7, 0.6, 0.5, 0.3, 0.4]

1. **Threshold = 1.5**

2. Multiply all inputs with its weights

- x1 * w1 = 1 * 0.7 = 0.7
- x2 * w2 = 0 * 0.6 = 0
- x3 * w3 = 1 * 0.5 = 0.5
- x4 * w4 = 0 * 0.3 = 0
- x5 * w5 = 1 * 0.4 = 0.4

3. Sum all the results:

- 0.7 + 0 + 0.5 + 0 + 0.4 = 1.6 (The Weighted Sum)

4. Activate the Output:

- Return true if the sum > 1.5("Yes I will go to the Concert")

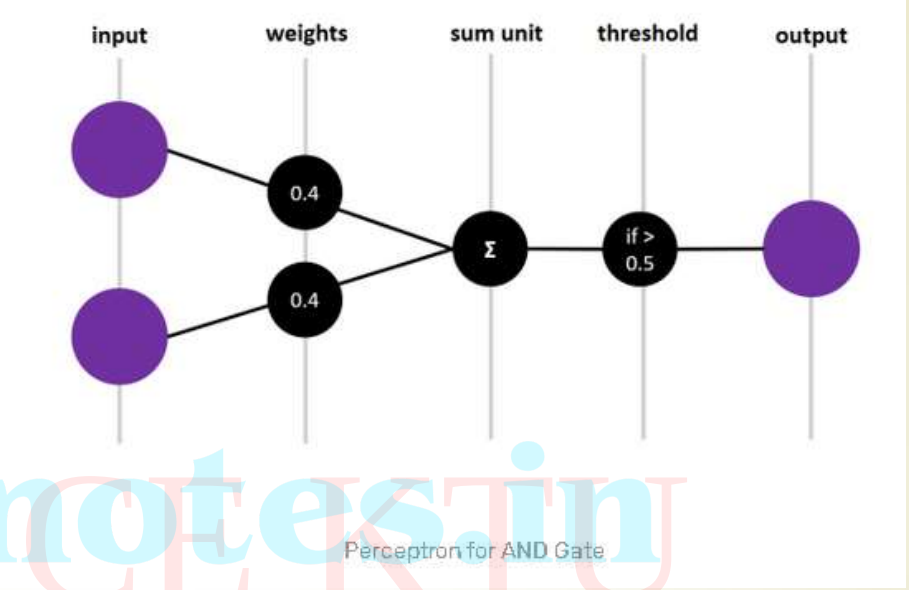
Knacles.in KNACLES

- If the weather weight is 0.6 for you, it might different for someone else. A higher weight means that the weather is more important to them.
- If the threshold value is 1.5 for you, it might be different for someone else. A lower threshold means they are more wanting to go to the concert.
- A Perceptron is often used to classify data into two parts.
- A Perceptron is also known as a **Linear Binary Classifier**.

KRACKEN.IO

AND gate problem

X_1	X_2	Y
0	0	0
0	1	0
1	0	0
1	1	1



Perceptron for AND Gate

Ran 1=0.9,w2=0.9

Threshold= 0.5

Kruncotes.in

Round 1

1st instance to the perceptron x1=0,x2=0

Weighted sum=0

Sum,0<0.5 , then the output is 0, it will not update the weight because there is no error in this case.

- **2nd instance to the perceptron x1=0,x2=1**

Weighted sum=0.9

Activation unit return 1, because 0.9>0.5

- output of this instance should be 0. This instance is not predicted correctly. That's why, we will update weights based on the error.
- ε = actual – prediction = 0 – 1 = -1
- Learning rate would be 0.5, We will add error times learning rate value to the weights
- $w_1 = w_1 + a * \varepsilon = 0.9 + 0.5 * (-1) = 0.9 - 0.5 = 0.4$
- $w_2 = w_2 + a * \varepsilon = 0.9 + 0.5 * (-1) = 0.9 - 0.5 = 0.4$
- **3rd instance. $x_1 = 1$ and $x_2 = 0$.**
- Sum unit: $\Sigma = x_1 * w_1 + x_2 * w_2 = 1 * 0.4 + 0 * 0.4 = 0.4$

KRACLES.in

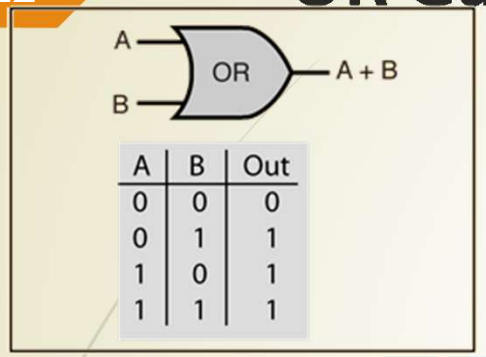
- Activation unit will return 0 this time because output of the sum unit is 0.5 and it is less than 0.5. We will not update weights.
- **4rd instance. $x_1 = 1$ and $x_2 = 1$.**
- sum unit: $\Sigma = x_1 * w_1 + x_2 * w_2 = 1 * 0.4 + 1 * 0.4 = 0.8$
- Activation unit will return 1 because output of the sum unit is 0.8 and it is greater than the threshold value 0.5. Its actual value should 1 as well. This means that 4th instance is predicted correctly. We will not update anything.
- **Round 2**
- 1st instance. $x_1 = 0$ and $x_2 = 0$.
- Sum unit: $\Sigma = x_1 * w_1 + x_2 * w_2 = 0 * 0.4 + 0 * 0.4 = 0.4$
- Activation unit will return 0 because sum unit is 0.4 and it is less than the threshold value 0.5. The output of the 1st instance should be 0 as well. This means that the instance is classified correctly.

Rnotes.in

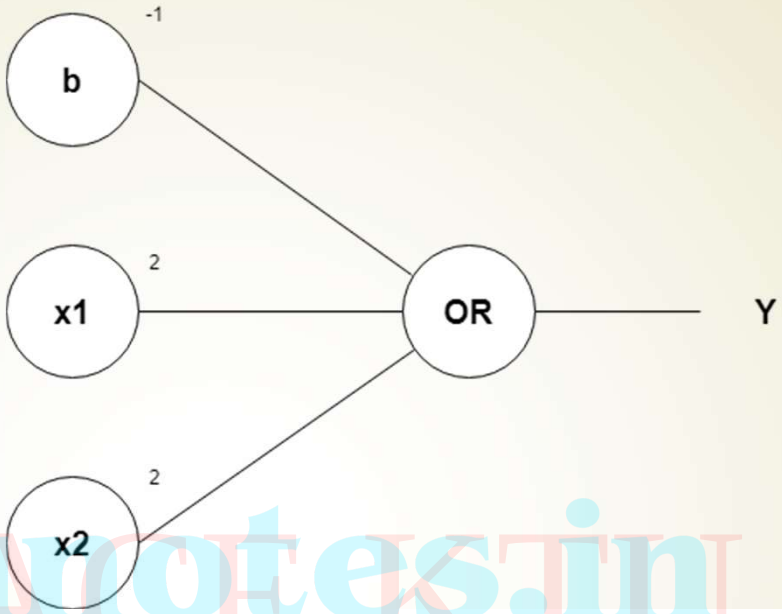
- 2nd instance. $x_1 = 0$ and $x_2 = 1$.
- Sum unit: $\Sigma = x_1 * w_1 + x_2 * w_2 = 0 * 0.4 + 1 * 0.4 = 0.4$
- Activation unit will return 0 because sum unit is less than the threshold 0.5. Its output should be 0 as well. This means that it is classified correctly and we will not update weights.
- for 3rd and 4th instances already for the current weight values in the previous round. They were classified correctly.

Krnotes.in

OR Gate



A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1



Kraotes.in

Row 1

- From w1x1+w2x2+b, initializing w1, w2, as 1 and b as –1, we get;

x1(1)+x2(1)–1

- Passing the first row of the OR logic table (x1=0, x2=0), we get;

0+0–1 = –1

From the Perceptron rule, if Wx+b≤0, then y`=0. Therefore, this row is correct.

Row 2

•Passing (x1=0 and x2=1), we get;

0+1–1 = 0

•From the Perceptron rule, if Wx+b <= 0, then y`=0. Therefore, this row is incorrect.

•So we want values that will make inputs x1=0 and x2=1 give y` a value of 1. If we change w2 to 2, we have;

0+2–1 = 1

•From the Perceptron rule, this is correct for both the row 1 and 2.

Row 3

•Passing (x1=1 and x2=0), we get;

1+0–1 = 0

•From the Perceptron rule, if Wx+b <= 0, then y`=0. Therefore, this row is incorrect.

•Since it is similar to that of row 2, we can just change w1 to 2, we have;

2+0–1 = 1

Row 4

•Passing (x1=1 and x2=1), we get;

2+2–1 = 3

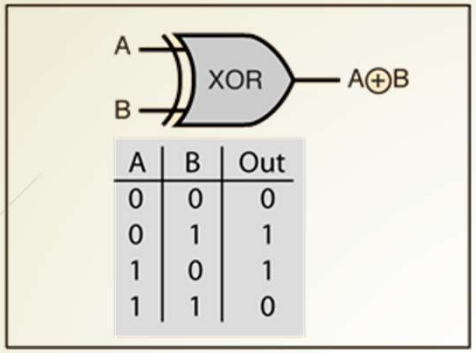
•Again, from the perceptron rule, this is still valid.

Therefore, we can conclude that the model to achieve an OR gate, using the Perceptron algorithm is;

2x1+2x2–1

Kraotes.in

XOR Gate



A	B	Out
0	0	0
0	1	1
1	0	1
1	1	0

The boolean representation of an XOR gate is;

x1x`2 + x`1x2

We first simplify the boolean expression

x`1x2 + x1x`2 + x`1x1 + x`2x2

x1(x`1 + x`2) + x2(x`1 + x`2)

(x1 + x2)(x`1 + x`2)

(x1 + x2)(x1x2)`

we can say that the XOR gate consists of an OR gate (x1 + x2), a NAND gate and an AND gate

Kraotes.in KRACE KTU

SINGLE LAYER PERCEPTRON,

- In the single layer network, a set of inputs is directly mapped to an output by using a generalized variation of a linear function.
- This simple instantiation of a neural network is also referred to as the perceptron.
- In multi-layer neural networks, the neurons are arranged in layered fashion, in which the input and output layers are separated by a group of hidden layers.
- This layer-wise architecture of the neural network is also referred to as a feed-forward network.
- The simplest neural network is referred to as the **perceptron**.

KRANOTES.in

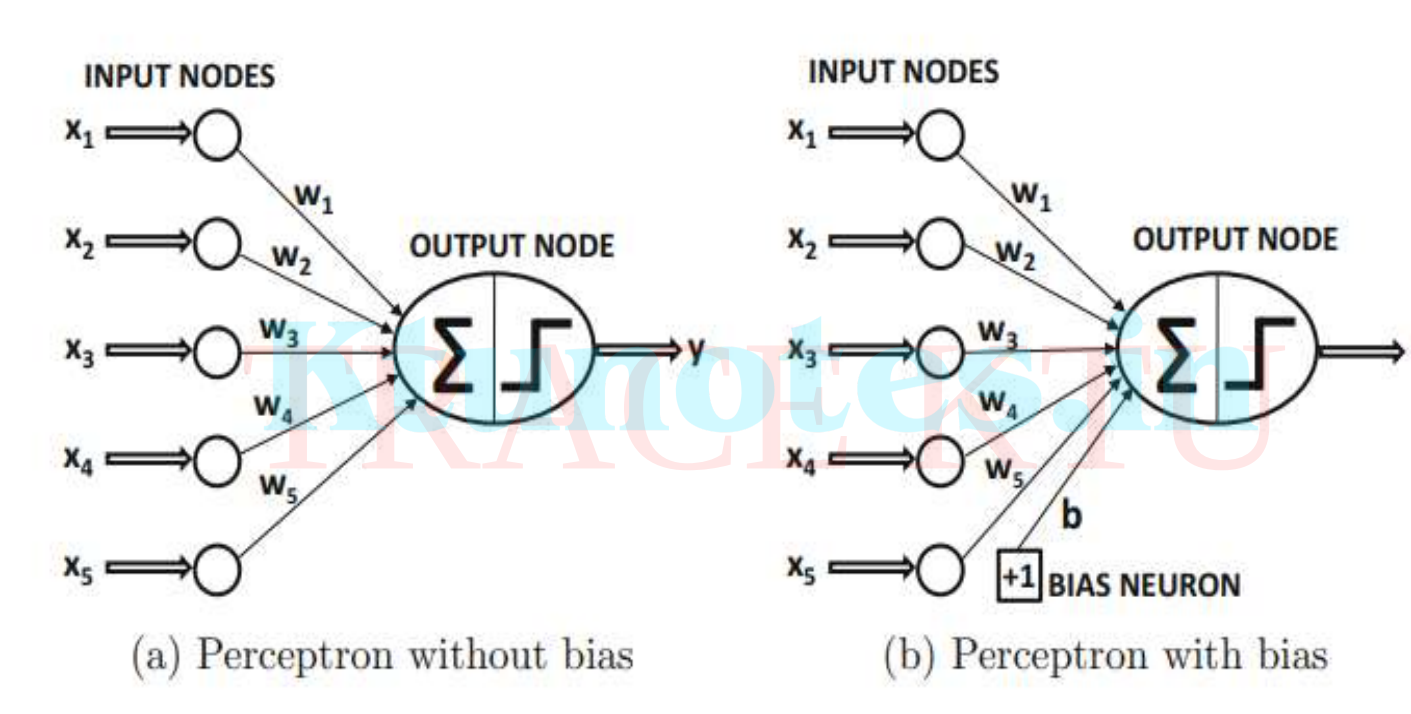


Fig:The basic architecture of the perceptron

- This neural network contains a **single input layer and an output node**.

- Each training instance is of the form (X,y), where each X = [x1,...xd] contains d feature variables, and y ∈ {−1, +1} contains the observed value of the binary class variable.

- By "observed value" we refer to the fact that it is given to us as a part of the training data, and our goal is to predict the class variable for cases in which it is not observed.
- The input layer contains d nodes that transmit the d features X = [x1 ...xd] with edges of weight W = [w1 ..wd] to an output node.
- The linear function is computed at the output node.

$$\overline{W} \cdot \overline{X} = \sum_{i=1}^{d} w_i x_i$$

- Subsequently, the sign of this real value is used in order to predict the dependent variable of X. Therefore, the prediction ˆy is computed as follows:

$$\hat{y} = \text{sign}\{\overline{W} \cdot \overline{X}\} = \text{sign}\{\sum_{j=1}^{d} w_j x_j\} \quad (1.1)$$

- The sign func for binary classification
- The error of the prediction is therefore E(X) = y − y^
- the perceptron contains two layers, although the input layer does not perform any computation and only transmits the feature values.
- The input layer is not included in the count of the number of layers in a neural network. Since the perceptron contains a single computational layer, it is considered a single-layer network.

KRAnotes.in

There is an invariant part of the prediction, which is referred to as the bias.

- The binary class distribution is highly imbalanced
- We need to incorporate an additional bias variable b that captures this invariant part of the prediction:

$$\hat{y} = \text{sign}\{\overline{W} \cdot \overline{X} + b\} = \text{sign}\{\sum_{j=1}^{d} w_j x_j + b\} \quad (1.2)$$

- **Perceptron al**
- The perceptro number of misclassificati ctness guarantees of the learning algorithm

Knacks.in

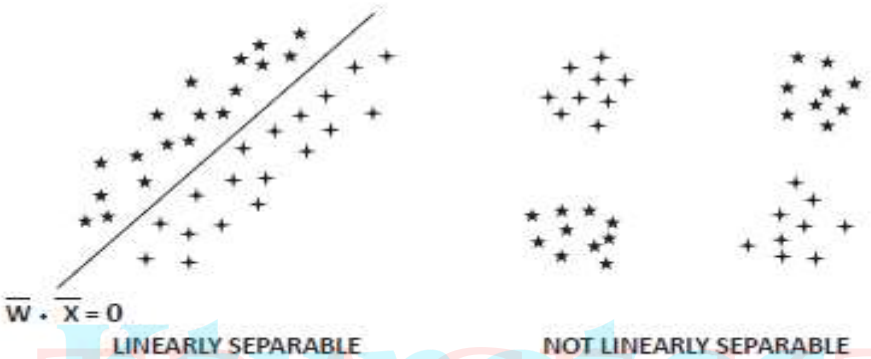
goal of the perceptron algorithm in least-squares form with respect to all training instances in a data set D containing feature-label pairs:

$$\text{Minimize}_{\overline{W}} L = \sum_{(\overline{X},y)\in\mathcal{D}} (y - \hat{y})^2 = \sum_{(\overline{X},y)\in\mathcal{D}} \left(y - \text{sign}\{\overline{W} \cdot \overline{X}\}\right)^2$$

- This type of minimization objective function is also referred to as a **loss function**

Kruotes.in

Where does the Perceptron Fail?



Krucoles.in

- The perceptron fails at similar problems as a linear SVM
  - **Classical solution:** Feature engineering with Radial Basis Function network $\Rightarrow$ Similar to kernel SVM and good for noisy data
  - **Deep learning solution:** Multilayer networks with non-linear activations $\Rightarrow$ Good for data with a lot of structure

Multilayer Perceptron(MLP)

- **Deep feedforward networks**, also often called **feedforward neural networks**,or **multilayer perceptrons** (MLPs),
- A feedforward network defines a mapping $y = f(x; \vartheta)$ and learns the value of the parameters ϑ that result in the best function approximation.
- There are no **feedback** connections in which outputs of the model are fed back into itself
- When feedforward neural networks are extended to include feedback connections, they are called **recurrent neural networks**
- The model is associated with a directed acyclic graph describing how the functions are composed together. For example, we might have three functions $f(1)$, $f(2)$, and $f(3)$ connected in a chain, to form $f(x) = f(3)(f(2)(f(1)(x)))$.
- These chain structures are the most commonly used structures of neural networks. In this case, $f(1)$ is called the **first layer** of the network, $f(2)$ is called the **second layer**, and so on.

- Multilayer neural networks contain more than one computational layer
- The perceptron contains an input and output layer, of which the output layer is the only computation performing layer
- The input layer transmits the data to the output layer, and all computations are completely visible to the user
- multilayer neural networks is referred to as *feed-forward* networks, because successive layers feed into one another in the forward direction from input to output.
- "perceptron" to refer to the basic unit of a neural network
- to use logistic units (with sigmoid activation) and piecewise/fully linear units as building blocks of these models.
- considered a composition function $f(g(\cdot))$.

TRACE R1U

- A multilayer network evaluates compositions of functions computed at individual nodes.
- A path of length 2 in the neural network in which the function $f(\cdot)$ follows $g(\cdot)$ can be
- if $g1(\cdot), g2(\cdot) \ldots gk(\cdot)$ are the functions
- computed in layer m, and a particular layer-$(m+1)$ node computes $f(\cdot)$, then the composition function computed by the layer-$(m + 1)$ node in terms of the layer-m inputs is $f(g1(\cdot), \ldots gk(\cdot))$.
- The use of nonlinear activation functions is the key to increasing the power of multiple layers.

Kruotes.in

- A deep neural network containing tens of layers can often be described in a few hundred lines of code.
- All the learning of the weights is done automatically by the *backpropagation algorithm* that uses dynamic programming to work out the complicated parameter update steps of the underlying computational graph
- length of the chain gives the **depth** of the model. It is from this terminology that the name "deep learning" arises. The final layer of a feedforward network is called the **output layer**.
- During neural network training, we drive $f(x)$ to match $f*(x)$.

Krnotes.in

The training data provides us with noisy, approximate examples of $f*(x)$ evaluated

- at different training points. Each example x is accompanied by a label $y \approx f*(x)$.the learning algorithm must decide how to use these layers to best implement an approximation of $f*$.
- Because the training data does not show the desired output for each of these layers, these layers are called **hidden layers**.
- Finally, these networks are called *neural* because they are loosely inspired by neuroscience. Each hidden layer of the network is typically vector-valued. The dimensionality of these hidden layers determines the **width** of the model

TRAORE.fr

- The strategy of deep learning is to learn ϕ
- $y = f(x; \theta, w) = \phi(x; \theta)^T w$.
- parameters w that map from ϕ(x) to the desired output
- parametrize the representation as ϕ(x; θ)and use the optimization algorithm to find the θ that corresponds to a good representation
- Feedforward networks (MLPs)are the application of this principle to learning deterministic mappings from x to y that lack feedback connections

Krunotes.in

Example: Learning XOR

- The XOR function ("exclusive or") is an operation on two binary values, $x1$ and $x2$. When exactly one of these binary values is equal to 1, the XOR function returns 1,Otherwise, it returns 0.
- The XOR function provides the target function $y = f*(x)$ that we want to learn.
- Our model provides a function $y = f(x; \theta)$ and our learning algorithm will adapt the parameters θ to make f as similar as possible to $f*$.
- four points X = {[0, 0]^T, [0, 1]^T,[1, 0]^T, and [1, 1]^T}.
- We can treat this problem as a regression problem and use a **mean squared error loss** function.
- Evaluated on our whole training set, the MSE loss function is

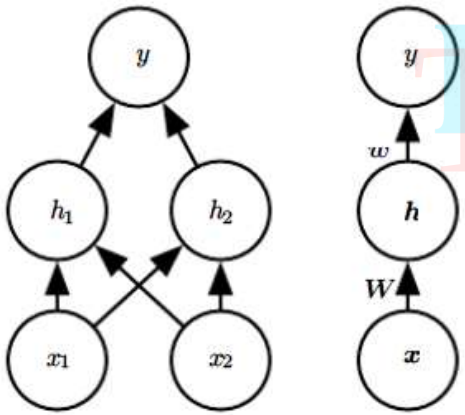
$$J(\boldsymbol{\theta}) = \frac{1}{4} \sum_{\boldsymbol{x} \in \mathbb{X}} (f^*(\boldsymbol{x}) - f(\boldsymbol{x}; \boldsymbol{\theta}))^2 . \qquad (6.1)$$

TRACER.IN

- Suppose that we choose a linear model, with θ consisting of w and b. Our model is defined to be

$$f(\boldsymbol{x}; \boldsymbol{w}, b) = \boldsymbol{x}^\top \boldsymbol{w} + b. \qquad (6.2)$$

- We can minimize $J(\theta)$ in closed form with respect to w and b using the normal equations.



- This feedforward network has a vector of hidden units h that are computed by a funct $f^{(1)}(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c})$.
- The values of these hidden units are then used as the input for a second layer. The second layer is the output layer of the network.
- The output layer is still just a linear regression model, but now it is applied to h rather than to x .

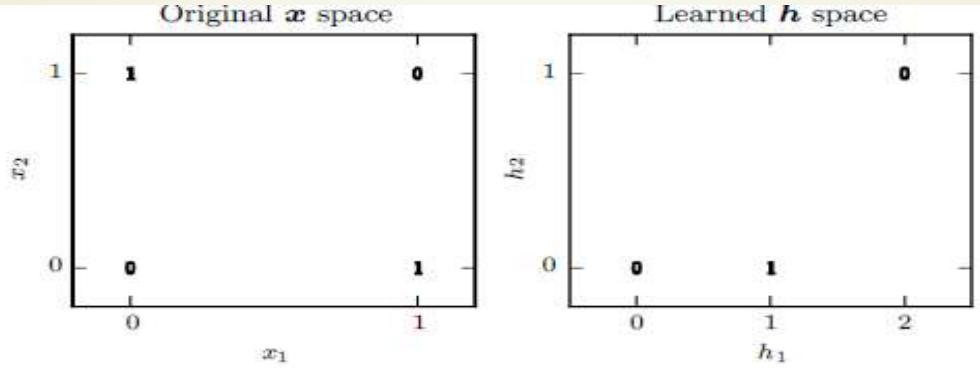


Figure 6.1: Solving the XOR problem by learning a representation. The bold numbers printed on the plot indicate the value that the learned function must output at each point. *(Left)*A linear model applied directly to the original input cannot implement the XOR function. When $x_1 = 0$, the model's output must increase as x_2 increases. When $x_1 = 1$, the model's output must decrease as x_2 increases. A linear model must apply a fixed coefficient w_2 to x_2. The linear model therefore cannot use the value of x_1 to change the coefficient on x_2 and cannot solve this problem. *(Right)*In the transformed space represented by the features extracted by a neural network, a linear model can now solve the problem. In our example solution, the two points that must have output 1 have been collapsed into a single point in feature space. In other words, the nonlinear features have mapped both $\boldsymbol{x} = [1, 0]^\top$ and $\boldsymbol{x} = [0, 1]^\top$ to a single point in feature space, $\boldsymbol{h} = [1 , 0]^\top$. The linear model can now describe the function as increasing in h_1 and decreasing in h_2. In this example, the motivation for learning the feature space is only to make the model capacity greater so that it can fit the training set. In more realistic applications, learned representations can also help the model to generalize.

- The network now contains two functions chained together $h = f^{(1)}(x; W, c)$ and $y = f^{(2)}(h; w, b)$, with the complete model being $f(x; W, c, w, b) = f^{(2)}(f^{(1)}(x))$.

- where W provides the weights of a linear transformation and c the biases.

- affine transformation from a vector x to a vector h, so an entire vector of bias parameters is needed.
- The activation function g is typically chosen to be a function that is applied element-wise,

$$h = g(W^\top x + c)$$

- In modern neural networks,the default recommendation is to use the **rectified linear unit** or ReLU (Jarrett *et al.*, 2009; Nair and Hinton, 2010; Glorot *et al.*, 2011a) defined by the activation function $g(z) = \max\{0, z\}$

KTUNOTES.IN

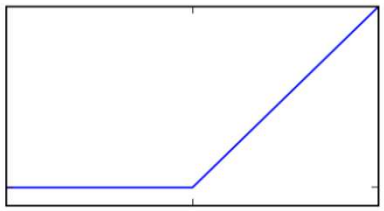
$$f(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \boldsymbol{w}^\top \max\{0, \boldsymbol{W}^\top \boldsymbol{x} + \boldsymbol{c}\} + b. \tag{6.3}$$

We can now specify a solution to the XOR problem. Let

$$\boldsymbol{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \tag{6.4}$$

$$\boldsymbol{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \tag{6.5}$$

Kraclets.in KRACLETS



- The rectified linear activation function. This activation function is the default activation function recommended for use with most feedforward neural networks. Applying this function to the output of a linear transformation yields a nonlinear transformation.
- However, the function remains very close to linear, in the sense that is a piecewise linear function with two linear pieces. Because rectified linear units are nearly linear, they preserve many of the properties that make linear models easy to optimize with gradient based methods. They also preserve many of the properties that make linear models generalize well
- . Much as a Turing machine's memory needs only to be able to store 0 or 1 states, we can build a universal function approximator from rectified linear functions

$$\boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \tag{6.6}$$

and $b = 0$.

We can now walk through the way that the model processes a batch of inputs. Let $\boldsymbol{X}$ be the design matrix containing all four points in the binary input space, with one example per row:

$$\boldsymbol{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}. \tag{6.7}$$

KTUCETRIP

The first step in the neural network is to multiply the input matrix by the first layer's weight matrix:

$$\boldsymbol{XW} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}. \tag{6.8}$$

Next, we add the bias vector $\boldsymbol{c}$, to obtain

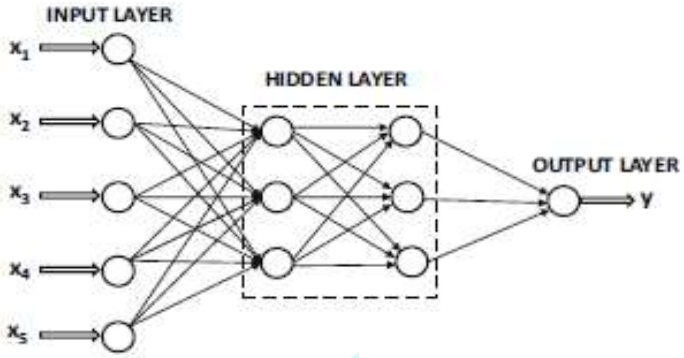
$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}. \tag{6.9}$$

In this space, all of the examples lie along a line with slope 1. As we move along this line, the output needs to begin at 0, then rise to 1, then drop back down to 0. A linear model cannot implement such a function. To finish computing the value of $\boldsymbol{h}$ for each example, we apply the rectified linear transformation:

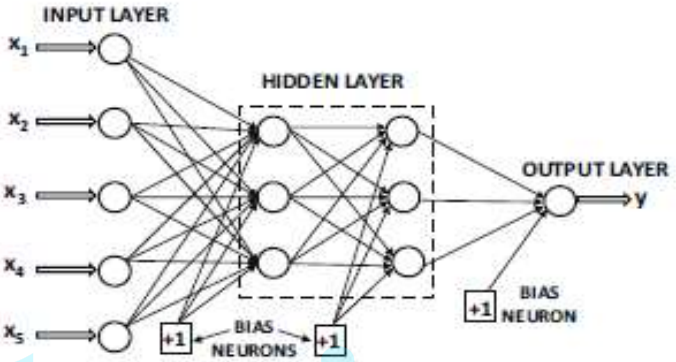
$$\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}. \tag{6.10}$$

Krutes.in

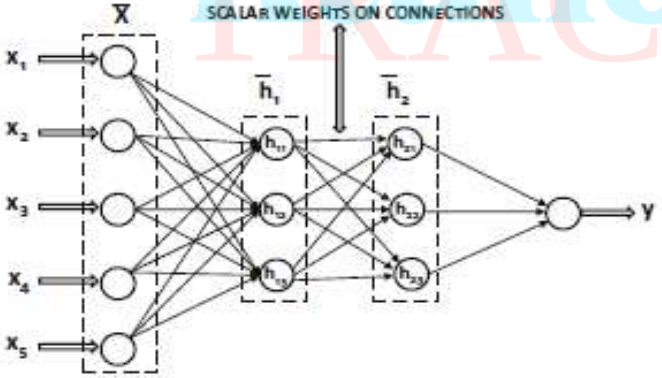
The neural network has obtained the correct answer for every example in the batch.



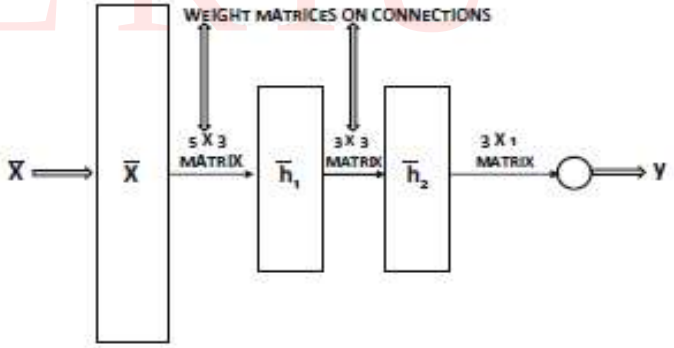
(a) No bias neurons



(b) With bias neurons



(c) Scalar notation and architecture



(d) Vector notation and architecture

KRACLES.in

Activation functions

- Why Do We Need Activation Functions?

• An activation function Φ(v) in the output layer can control the nature of the output (e.g., probability value in [0, 1])

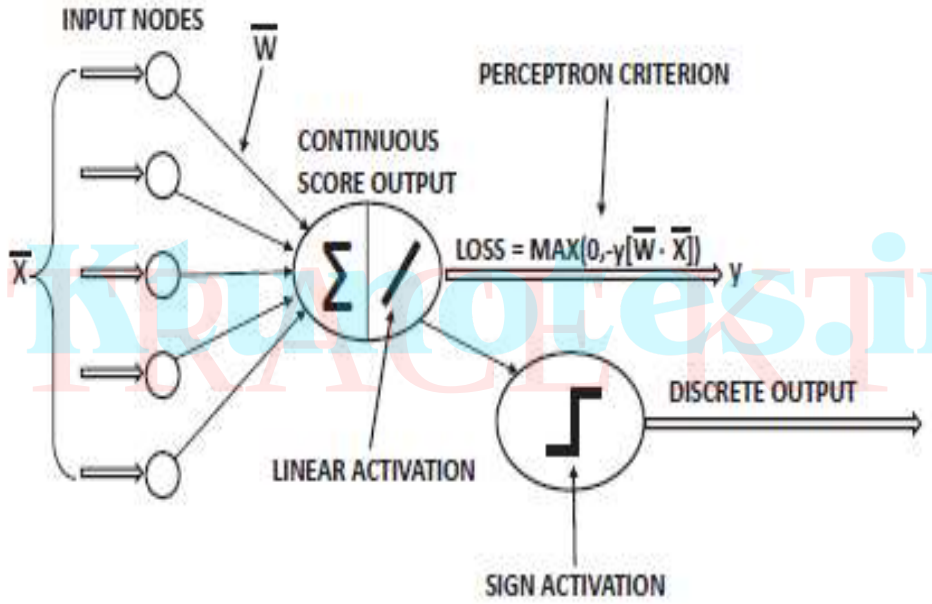
• In multilayer neural networks, activation functions bring non-linearity into hidden layers, which increases the complexity of the model.

– A neural network with any number of layers but only linear activations can be shown to be equivalent to a single-layer network.

KTUNotes.in

- Activation functions required for inference may be different from those used in loss functions in training.
- Perceptron uses sign function $\Phi(v) = \text{sign}(v)$ for prediction but does not use any activation for computing the perceptron criterion (during training).

Perceptron: Activation Functions



Kpaoles.in

Identity Activation

- Identity activation $\Phi(v) = v$ is often used in the output layer, when the outputs are real values.
- For a single-layer network, if the training pair is $(\overline{X}, y)$, the output is as follows:

$$\hat{y} = \Phi(\overline{W} \cdot \overline{X}) = \overline{W} \cdot \overline{X}$$

KRAnotes.in

- Use of the squared loss function $(y - \hat{y})^2$ leads to the *linear regression* model with numeric outputs and *Widrow-Hoff learning* with binary outputs.
- Identity activation can be combined with various types of loss functions (e.g., perceptron criterion) even for discrete outputs.

Sigmoid Activation

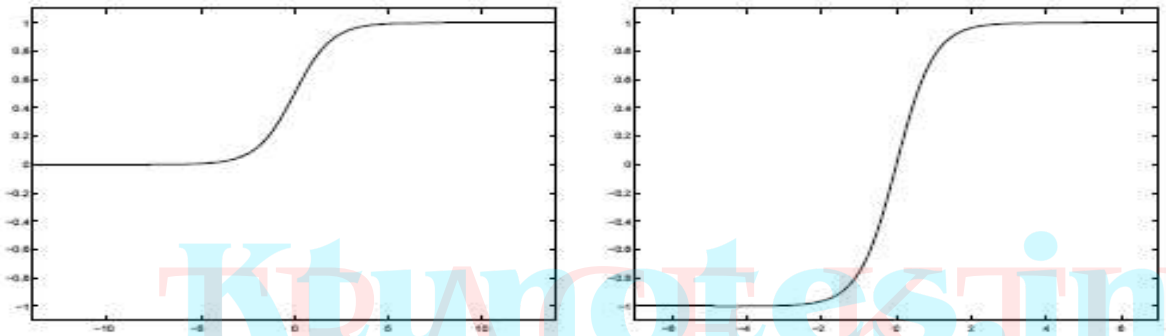
- Sigmoid activation is defined as $\Phi(v) = 1/(1 + \exp(-v))$.
- For a training pair $(\overline{X}, y)$, one obtains the following prediction in a single-layer network:

$$\hat{y} = 1/(1 + \exp(-\overline{W} \cdot \overline{X}))$$

Kraoles.in

- Prediction is the *probability* that class label is $+1$.
- Paired with *logarithmic loss*, which $-\log(\hat{y})$ for positive instances and $-\log(1 - \hat{y})$ for negative instances.
- Resulting model is *logistic regression*.

Tanh Activation



(a) Sigmoid (b) Tanh

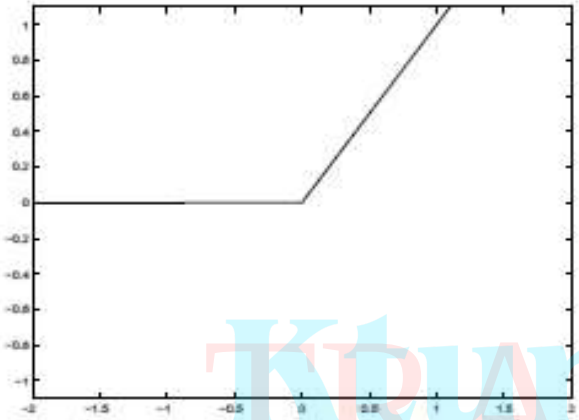
Kruetes.in

- The tanh activation is a scaled and translated version of sigmoid activation.

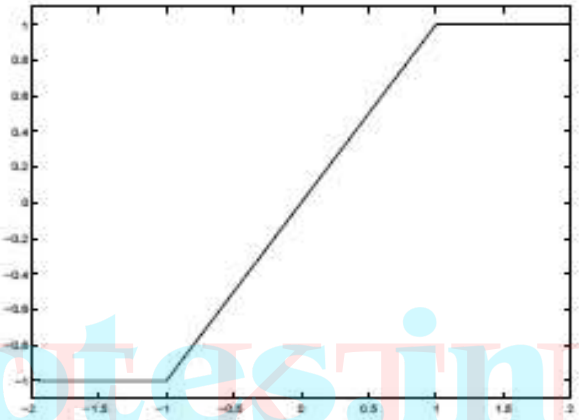
$$\tanh(v) = \frac{e^{2v} - 1}{e^{2v} + 1} = 2 \cdot \text{sigmoid}(2v) - 1$$

- Often used in hidden layers of multilayer networks

Piecewise Linear Activation Functions



(a) ReLU
$\Phi(v) = \max\{v, 0\}$



(b) Hard Tanh
$\Phi(v) = \max\{\min[v, 1], -1\}$

- Piecewise linear activation functions are easier to train than their continuous counterparts.

Softmax Activation Function

- All activation functions discussed so far map scalars to scalars.
- The softmax activation function maps vectors to vectors.
- Useful in mapping a set of real values to probabilities.
  - Generalization of sigmoid activation, which is used in *multiway* logistic regression.

KRACLES.in

- Different types of nonlinear functions such as the sign, sigmoid, or hyperbolic tangents may be used in various layers.
- notation **Φ to denote the activation function**:
- $\hat{y} = \Phi(W \cdot X)$
- Therefore, a neuron really computes two functions within the node, which is why we have incorporated the summation symbol **Σ** as well as the activation symbol Φ within a neuron.
- The break-up of the neuron computations into two separate values is shown in Figure

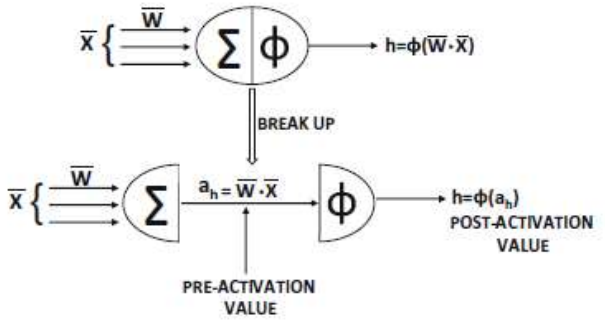


Figure 1.7: Pre-activation and post-activation values within a neuron

The value computed before applying the activation function $\Phi(\cdot)$ will be referred to as the *pre-activation value*,

- whereas the value computed after applying the activation function is referred to as the *post-activation value*. The output of a neuron is always the post-activation value.
- The most basic activation function $\Phi(\cdot)$ is the identity or linear activation, which provides no nonlinearity:
- $\Phi(v) = v$
- The linear activation function is often used in the output node, when the target is a real value.

Rnotes.in

The classical activation functions that were used early in the development of neural networks were the sign, sigmoid, and the hyperbolic tangent functions:

$$\Phi(v) = \text{sign}(v) \text{ (sign function)}$$

$$\Phi(v) = \frac{1}{1+e^{-v}} \text{ (sigmoid function)}$$

$$\Phi(v) = \frac{e^{2v}-1}{e^{2v}+1} \text{ (tanh function)}$$

RGPVNOTES.in

- while the perceptron uses the sign function for prediction, the perceptron criterion in training only requires linear activation.
- The sigmoid activation outputs a value in $(0, 1)$, which is helpful in performing computations that should be interpreted as probabilities.
- it is also helpful in creating probabilistic outputs and constructing loss functions derived from maximum-likelihood models.
- The tanh function has a shape similar to that of the sigmoid function, except that it is horizontally re-scaled and vertically translated/re-scaled to $[-1, 1]$.
- $\tanh(v) = 2 \cdot \text{sigmoid}(2v) - 1$
- The tanh function is preferable to the sigmoid when the outputs of the computations are desired to be both positive and negative

KNACLER.in

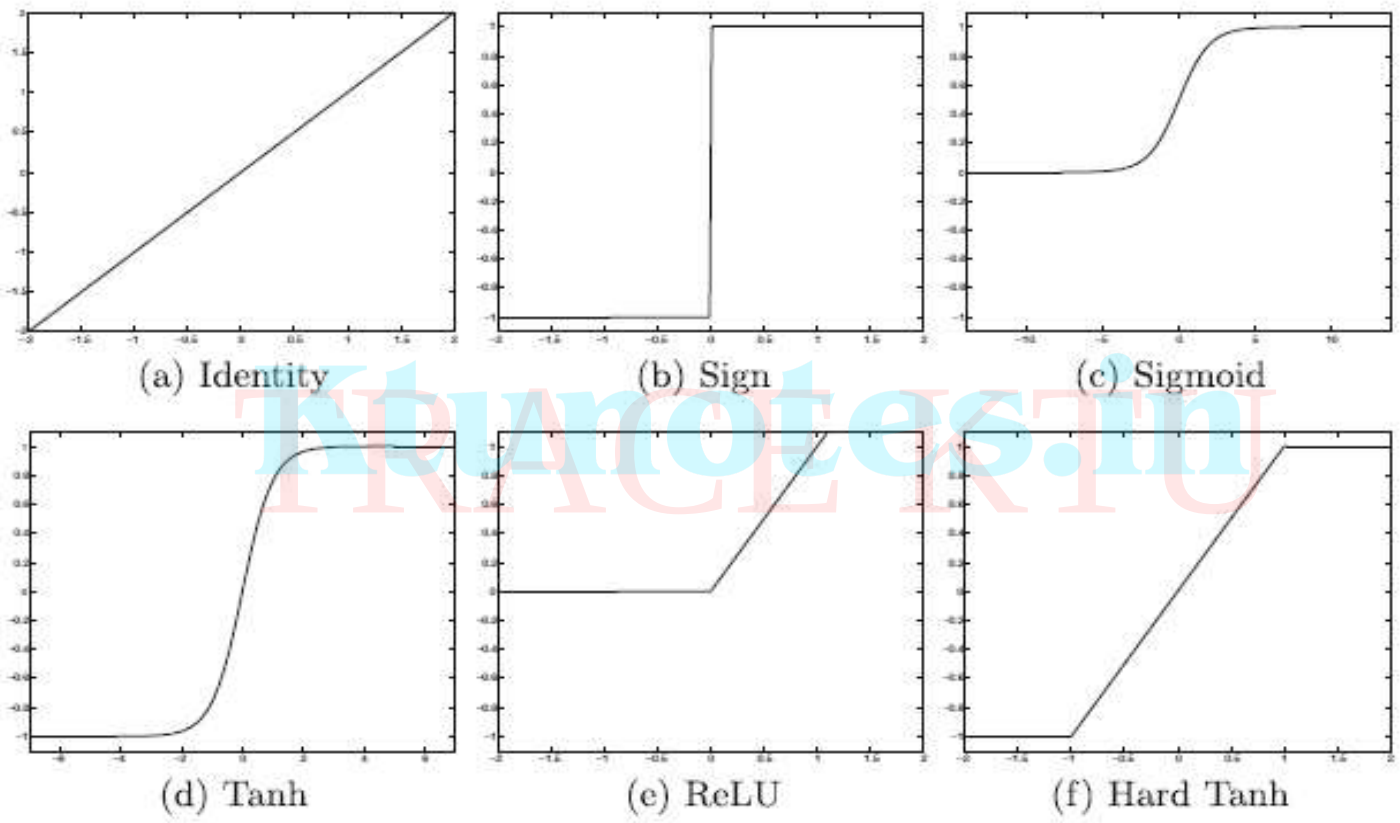


Figure 1.8: Various activation functions

Knarotes.in

- The **sigmoid and the tanh** functions have been the historical tools of choice for incorporating **nonlinearity** in the neural network.
- In recent years, however, a number of piecewise linear activation functions have become more popular:
- $\Phi(v) = \max\{v, 0\}$ (Rectified Linear Unit [ReLU])
- $\Phi(v) = \max\{\min[v, 1], -1\}$ (hard tanh)
- The **ReLU and hard tanh** activation functions have largely replaced the sigmoid and soft tanh activation functions in modern neural networks because of the ease in training multilayered neural networks with these activation functions

Kuracter.in

CST414
DEEP LEARNING
MODULE 1-PART 2

Ktunotes.in

SYLLABUS

Module-1 (Neural Networks)

Introduction to neural networks -Single layer perceptrons, Multi Layer Perceptrons (MLPs), **Representation Power of MLPs**, Activation functions - Sigmoid, Tanh, ReLU, Softmax. , **Risk minimization, Loss function, Training MLPs with backpropagation, Practical issues in neural network training - The Problem of Overfitting, Vanishing and exploding gradient problems, Difficulties in convergence, Local and spurious Optima, Computational Challenges. Applications of neural networks.**

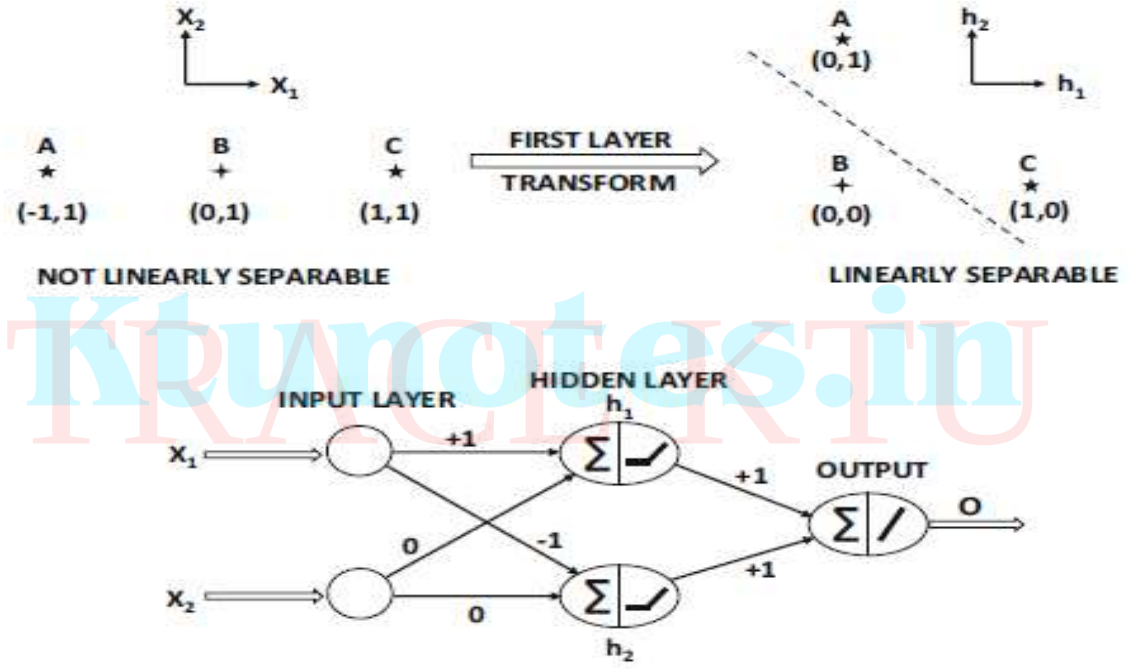
KTUAlerts.in

REPRESENTATION POWER OF MLP

- Representation power is related to ability of a neural network **to assign proper labels** to a particular instance and create well defined accurate decision boundaries for that class.
- a neural network is **a computational graph** that performs compositions of simpler functions to provide a more complex function
- power of deep learning arises from the fact that ***repeated* composition of multiple nonlinear functions** has significant expressive power.
- the *repeated composition of certain types of functions increases the representation power of the network, and therefore reduces the parameter space required for learning.*

KTUNotes.in

The Importance of Nonlinear Activation



Krunotes.in

Figure 1.14: The power of nonlinear activation functions in transforming a data set to linear separability

- A neural network with only linear activations will never be able to classify the training data perfectly because the points are not linearly separable.
- On the other hand, consider a situation in which the hidden units have ReLU activation, and they learn the two new features $h1$ and $h2$, which are as follow:

$$h_1 = \max\{x_1, 0\}$$
$$h_2 = \max\{-x_1, 0\}$$

KRAnotes.in

- Note that these goals can be achieved by using appropriate weights from the input to hidden layer, and also applying a ReLU activation unit.
- The latter achieves the goal of thresholding negative values to 0.

- The coordinates of the three points in the 2-dimensional hidden layer are {(1, 0), (0, 1), (0, 0)}. It is immediately evident that the two classes become linearly separable in terms of the new hidden representation.
- In a sense, the task of the first layer was *representation learning* to enable the solution of the problem with a linear classifier.
- The key point is that the use of the **nonlinear ReLU function** is crucial in ensuring this linear separability.
- *Activation functions enable nonlinear mappings of the data, so that the embedded points can become linearly separable.*

- if both the weights from hidden to output layer are set to 1 with a linear activation function, the output *O* will be defined as follows:
- $O = h1 + h2$
- This simple linear function separates the two classes because it always takes on the value of 1 for the two points labeled '*' and takes on 0 for the point labeled '+'.
- Therefore, much of the power of neural networks is hidden in the use of activation functions

KRACOLES.IN

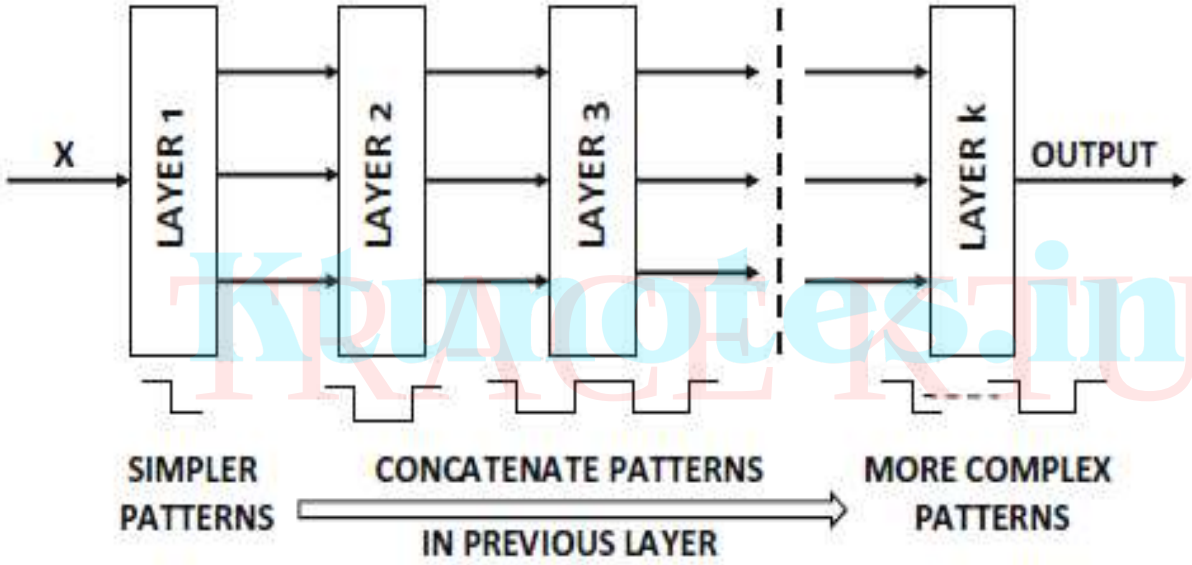


Figure 1.15: Deeper networks can learn more complex functions by composing the functions learned in earlier layers.

- Activation functions enable nonlinear transformations of the data, that become increasingly powerful with multiple layers.

- Sequence of nonlinear activations imposes a specific type of structure on the learned model, whose power increases with the depth of the sequence (i.e., number of layers in the neural network)

TRACE KTU

- **Advantages of sigmoid**
- The main reason why we use sigmoid function is because it exists between (0 to 1). Therefore, **it is especially used for models where we have to predict the probability as an output**.Since probability of anything exists only between the range of 0 and 1,
- In binary classification, also called logistic regression, the sigmoid function is used to predict the probability of a binary variable.
- σ(sigma) $\sigma(x) = \frac{1}{1 + e^{-x}}$
- E=2.781
- **Disadvantages of the Sigmoid Activation Function**
- One of the major disadvantages of using sigmoid is the problem of vanishing gradient. ...
- It saturates and kills gradients. ...
- It is computationally expensive because of the exponential term in it.

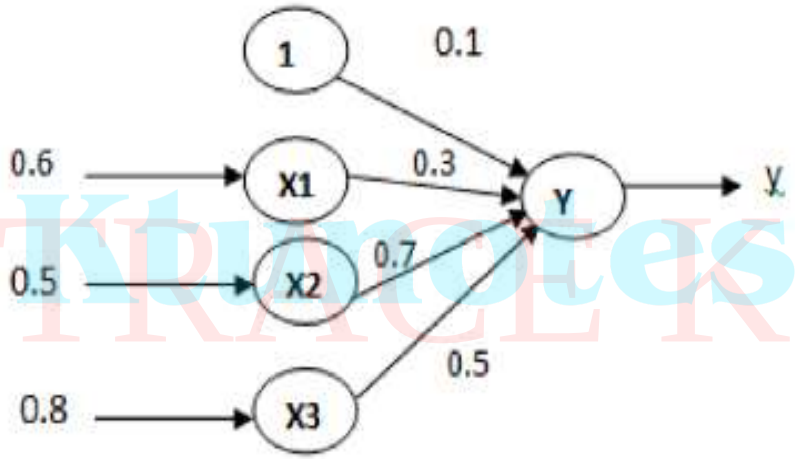
Knotes.in KNACE KTU

- **Advantages of ReLu**
- **The model trained with ReLU converged quickly and thus takes much less time when compared to models trained on the Sigmoid function**. The model performance is significantly better when trained with ReLU
- **It doesn't allow for the activation of all of the neurons at the same time**. i.e., if any input is negative, ReLU converts it to zero and doesn't allow the neuron to get activated. This means that only a few neurons are activated, making the network easy for computation.
- Disadvantages of ReLu
- The drawback of ReLU is that **they cannot learn on examples for which their activation is zero**. It usually happens if you initialize the entire neural network with zero and place ReLU on the hidden layers.
- ReLu is faster to compute than the sigmoid function,

KRACKER IO

Problem

Calculate the output of the following neuron Y if the activation function is a **binary sigmoid**



Kturoles.in
TRACE KTU

Assignment 1

- 1. Suppose you have a 3-dimensional input x = (x1, x2, x3) = (2, 2, 1) fully connected to 1 neuron which is in the hidden layer with activation function sigmoid. Calculate the output of the hidden layer neuron
- 2. Design a single layer perceptron to compute the **NAND (not-AND)** function. This function receives two binary-valued inputs **x1** and **x2**, and returns 0 if both inputs are 1, and returns 1 otherwise.
- 3. Suppose we have a fully connected, feed-forward network with no hidden layer, and 5 input units connected directly to 3 output units. Briefly explain why adding a hidden layer with 8 linear units does not make the network any more powerful.
- 4. Briefly explain one thing you would use a validation set for, and why you can't just do it using the test set.

KRACE KTU notes.in

- **Submit on or before 13th march 2023**

Loss Function

- The choice of the loss function is critical in defining the outputs in a way that is sensitive to the application at hand.
- For example, least-squares regression with numeric outputs requires a simple **squared loss of the form $(y - \hat{y})^2$** for a single training instance with target y and prediction $\hat{y}$.
- other types of loss like ***hinge loss*** for $y \in \{-1, +1\}$ and real-valued prediction $\hat{y}$ (with identity activation):
- $L = \max\{0, 1 - y \cdot \hat{y}\}$
- The hinge loss can be used to implement a learning method, which is referred to as a *support vector machine*

KRACOLES.in

- For probabilistic predictions, two different types of loss functions are used, depending on whether the prediction is binary or whether it is multiway:
- 1. **Binary targets (logistic regression):**
- In this case, it is assumed that the observed value y is drawn from $\{-1, +1\}$, and the prediction $\hat{y}$ is a an arbitrary numerical value on using the identity activation function.
- In such a case, the loss function for a single instance with observed value y and real-valued prediction $\hat{y}$ (with identity activation) is defined as follows:
- $L = \log(1 + \exp(-y \cdot \hat{y}))$
- This type of loss function implements a fundamental machine learning method, referred to as *logistic regression*
- one can use a sigmoid activation function to output $\hat{y} \in (0, 1)$, which indicates the probability that the observed value y is 1.

- 2. **Categorical targets:**
- In this case, if $\hat{y}1 \ldots \hat{y}k$ are the probabilities of the k classes and the rth class is the ground-truth class, then the loss function for a single instance is defined as follows:
- $L = -\log(\hat{y}r)$
- This type of loss function implements multinomial logistic regression, and it is referred to as the ***cross-entropy loss***.
- The key point to remember is that the nature of the output nodes, the activation function, and the loss function depend on the application at hand.
- For discrete-valued outputs, it is common to use softmax activation with cross entropy
- loss. For real-valued outputs, it is common to use linear activation with squared
- loss. Generally, cross-entropy loss is easier to optimize than squared loss.

Risk minimization

- The goal of a machine learning algorithm is to reduce the expected generalization error given by equation

$$J^*(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},\mathrm{y})\sim p_{\mathrm{data}}} L(f(\boldsymbol{x};\boldsymbol{\theta}), y).$$

- This quantity is known as the **risk**.
- the expectation is taken over the true underlying distribution *P*data.
- to convert a machine learning problem back into an optimization
- problem is to minimize the expected loss on the training set. This
- means replacing the true distribution $p(x, y)$ with the empirical distribution $\hat{p}(x, y)$ defined by the training set. We now minimize the **empirical risk**

TRACLES.in

$$\mathbb{E}_{\boldsymbol{x},\mathrm{y}\sim\hat{p}_{\mathrm{data}}(\boldsymbol{x},y)}[L(f(\boldsymbol{x};\boldsymbol{\theta}),y)] = \frac{1}{m}\sum_{i=1}^{m} L(f(\boldsymbol{x}^{(i)};\boldsymbol{\theta}),y^{(i)})$$

- where m is the number of training examples
- The training process based on minimizing this average training error is known as **empirical risk minimization.**
- empirical risk minimization is prone to overfitting
- The most effective modern optimization algorithms are based on gradient descent,
- In the context of deep learning, we rarely use empirical risk minimization

TRAINING MLPS WITH BACKPROPAGATION

- In the single-layer neural network, the training process is relatively straightforward because the **error (or loss function)** can be computed as a direct function of the weights,
- In the case of multi-layer networks, the problem is that the loss is a complicated composition function of the weights in earlier layers
- **The gradient of a composition function is computed using the backpropagation algorithm.**
- The backpropagation algorithm leverages the **chain rule of differential calculus**, which computes the error gradients in terms of summations of local-gradient products over the various paths from a node to the output.

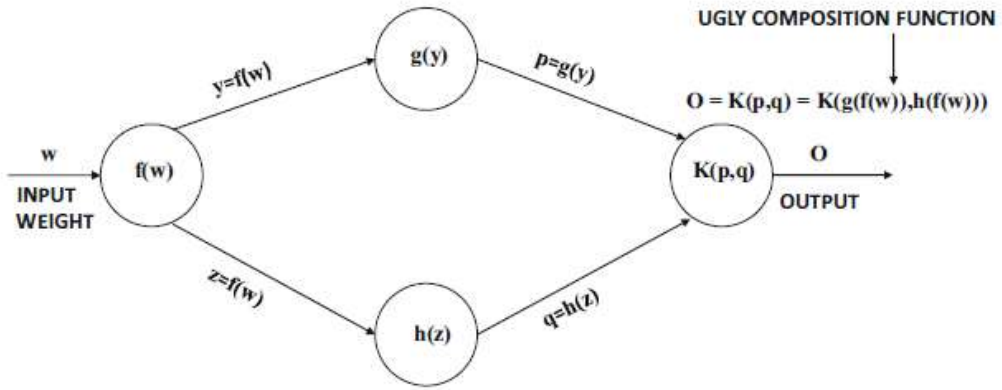
KRUCIALTECH

- The backpropagation algorithm is a direct application of dynamic programming.
- It contains two main phases, referred to as the *forward* and *backward* phases, respectively. **The forward phase** is required to compute the output values and the local derivatives at various nodes, and the **backward phase** is required to accumulate the products of these local values over all paths from the node to the output:
- 1. ***Forward phase:***
- In this phase, the inputs for a training instance are fed into the neural network. This results in a forward cascade of computations across the layers, using the current set of weights.
- The final predicted output can be compared to that of the training instance and the derivative of the loss function with respect to the output is computed.

KRACLES.in

- **2. *Backward phase***: The main goal of the backward phase is to learn the gradient of the loss function with respect to the different weights by using the chain rule of differential calculus.
- These gradients are used to update the weights. Since these gradients are learned in the backward direction, starting from the output node, this learning process is referred to as the backward phase

Krucotes.in



$$\frac{\partial o}{\partial w} = \frac{\partial o}{\partial p} \cdot \frac{\partial p}{\partial w} + \frac{\partial o}{\partial q} \cdot \frac{\partial q}{\partial w} \quad \text{[Multivariable Chain Rule]}$$

$$= \frac{\partial o}{\partial p} \cdot \frac{\partial p}{\partial y} \cdot \frac{\partial y}{\partial w} + \frac{\partial o}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial w} \quad \text{[Univariate Chain Rule]}$$

$$= \underbrace{\frac{\partial K(p,q)}{\partial p} \cdot g'(y) \cdot f'(w)}_{\text{First path}} + \underbrace{\frac{\partial K(p,q)}{\partial q} \cdot h'(z) \cdot f'(w)}_{\text{Second path}}$$

Figure 1.13: **Illustration of chain rule in computational graphs:** The products of node-specific partial derivatives along paths from weight w to output o are aggregated. The resulting value yields the derivative of output o with respect to weight w. Only two paths between input and output exist i

- Consider a sequence of hidden units $h1, h2, \ldots, hk$ followed by output o, with respect to which the loss function L is computed.
- assume that the **weight of the connection** from hidden unit hr to $hr+1$ is $\boldsymbol{w(hr,hr+1)}$.
- in the case that a single path exists from $h1$ to o, one can derive the gradient of the loss function with respect to any of these edge weights using the chain rule:

$$\frac{\partial L}{\partial w_{(h_{r-1},h_r)}} = \frac{\partial L}{\partial o} \cdot \left[\frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right] \frac{\partial h_r}{\partial w_{(h_{r-1},h_r)}} \quad \forall r \in 1 \ldots k$$

Knoldus

- A generalized variant of the chain rule, referred to **as the *multivariable chain rule*, computes the gradient in a computational graph, where more than one path might exist.**
- This is achieved by adding the composition along each of the paths from $h1$ to o.

$$\frac{\partial L}{\partial w_{(h_{r-1},h_r)}} = \underbrace{\frac{\partial L}{\partial o} \cdot \left[\sum_{[h_r,h_{r+1},\ldots h_k,o] \in \mathcal{P}} \frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right]}_{\text{Backpropagation computes } \Delta(h_r,o) = \frac{\partial L}{\partial h_r}} \frac{\partial h_r}{\partial w_{(h_{r-1},h_r)}}$$

- the path-aggregated term above [annotated **by $\Delta(hr, o) = \partial L/\partial hr$**] is aggregated over an exponentially increasing number of paths .
- A key point is that the **computational graph of a neural network does not have cycles**, it is possible to compute such an aggregation in a principled way in the backwards direction by first computing $\Delta(hk, o)$ for nodes hk closest to o, and then recursively computing these values for nodes in earlier layers in terms of the nodes in later layers.
- Furthermore, the value of $\Delta(o, o)$ for each output node is initialized as follows:

$$\Delta(o, o) = \frac{\partial L}{\partial o}$$

- The recursion for $\Delta(hr, o)$ can be derived using the multivariable chain rule:

$$\Delta(h_r, o) = \frac{\partial L}{\partial h_r} = \sum_{h:h_r \Rightarrow h} \frac{\partial L}{\partial h} \frac{\partial h}{\partial h_r} = \sum_{h:h_r \Rightarrow h} \frac{\partial h}{\partial h_r} \Delta(h, o)$$

- Consider a situation in which the edge joining *hr* to *h* has weight $w(hr,h)$,and let *ah* be the value computed in hidden unit *h* just *before* applying the activation function $\Phi(\cdot)$.
- we have $h = \Phi(ah)$, where *ah* is a linear combination of its inputs from earlier-layer units incident on *h*. Then, by the univariate chain rule, the following expression for $\partial h/\partial hr$ can be derived:

$$\frac{\partial h}{\partial h_r} = \frac{\partial h}{\partial a_h} \cdot \frac{\partial a_h}{\partial h_r} = \frac{\partial \Phi(a_h)}{\partial a_h} \cdot w_{(h_r,h)} = \Phi'(a_h) \cdot w_{(h_r,h)}$$

Krnotes.in

- which is repeated recursively in the backwards direction, starting with the output node. The corresponding updates in the backwards direction are as follows:

$$\Delta(h_r, o) = \sum_{h:h_r \Rightarrow h} \Phi'(a_h) \cdot w_{(h_r,h)} \cdot \Delta(h, o)$$

- needs to be repeated for each incoming edge into the node to compute the gradient with respect to all edge weights.

$$\frac{\partial h_r}{\partial w_{(h_{r-1},h_r)}} = h_{r-1} \cdot \Phi'(a_{h_r})$$

- **the key gradient that is backpropagated is the derivative with respect to *layer activations*, and the gradient with respect to the weights is easy to compute for any incident edge on the corresponding unit.**
- uses the variables in the hidden layers as the "chain" variables for the dynamic programming recursion.
- The pre-activation variables in a neuron are obtained after applying the linear transform as the intermediate variables. The pre-activation value of the hidden variable $h = \Phi(ah)$ is ah. The differences between the pre-activation and post-activation values within a neuron

$$\frac{\partial L}{\partial w_{(h_{r-1},h_r)}} = \underbrace{\frac{\partial L}{\partial o} \cdot \Phi'(a_o) \cdot \left[\sum_{[h_r,h_{r+1},\ldots h_k,o] \in \mathcal{P}} \frac{\partial a_o}{\partial a_{h_k}} \prod_{i=r}^{k-1} \frac{\partial a_{h_{i+1}}}{\partial a_{h_i}} \right]}_{\text{Backpropagation computes } \delta(h_r,o) = \frac{\partial L}{\partial a_{h_r}}} \underbrace{\frac{\partial a_{h_r}}{\partial w_{(h_{r-1},h_r)}}}_{h_{r-1}}$$

KRACE KTU

Here, we have introduced the notation $\delta(h_r, o) = \frac{\partial L}{\partial a_{h_r}}$ instead of $\Delta(h_r, o) = \frac{\partial L}{\partial h_r}$ for setting up the recursive equation. The value of $\delta(o, o) = \frac{\partial L}{\partial a_o}$ is initialized as follows:

$$\delta(o, o) = \frac{\partial L}{\partial a_o} = \Phi'(a_o) \cdot \frac{\partial L}{\partial o} \tag{1.29}$$

Then, one can use the multivariable chain rule to set up a similar recursion:

$$\delta(h_r, o) = \frac{\partial L}{\partial a_{h_r}} = \sum_{h:h_r \Rightarrow h} \overbrace{\frac{\partial L}{\partial a_h}}^{\delta(h,o)} \underbrace{\frac{\partial a_h}{\partial a_{h_r}}}_{\Phi'(a_{h_r}) w_{(h_r,h)}} = \Phi'(a_{h_r}) \sum_{h:h_r \Rightarrow h} w_{(h_r,h)} \cdot \delta(h, o) \tag{1.30}$$

This recursion condition is found more commonly in textbooks discussing backpropagation. The partial derivative of the loss with respect to the weight is then computed using $\delta(h_r, o)$ as follows:

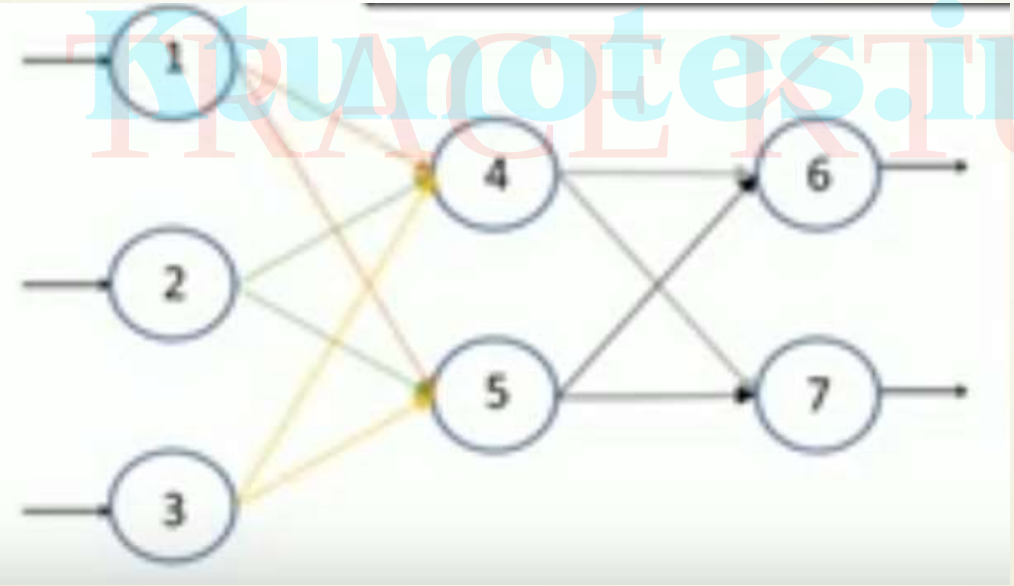
$$\frac{\partial L}{\partial w_{(h_{r-1},h_r)}} = \delta(h_r, o) \cdot h_{r-1} \tag{1.31}$$

KRAnotes.in

- As with the single-layer network, the process of updating the nodes is repeated to convergence by repeatedly cycling through the training data in epochs. A neural network may sometimes require thousands of epochs through the training data to learn the weights at the different nodes.

Derivation of Back propagation Algorithm

- In Gradient Descent, we consider all the points in calculating loss and derivative, while in Stochastic gradient descent, we use single point in loss function and its derivative randomly



Rnotes.in

- To derive the equation for updating weights in back propagation algorithm, we use Stochastic gradient descent rule.
- Stochastic gradient descent involves iterating through the training examples one at a time, for each training example **d** descending the gradient of the error ***Ed*** with respect to this single example.
- In other words, for each training example **d** every weight ***wji*** is updated by adding to it Δw_{ij}.
- That is,

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

KTUQUESTIONS.IN

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

- where ***Ed*** is the error on training example **d**, that is half the squared difference between the target output and the actual output over all output units in the network,

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

- Here outputs is the set of output units in the network, $\mathbf{t_k}$ is the target value of unit k for training example **d**, and $\mathbf{o_k}$ is the output of unit k given training example **d**.

Notation Used:

x_{ji} = the ith input to unit j

w_{ji} = the weight associated with the ith input to unit j

$net_j = \sum_i w_{ji} X_{ji}$ (the weighted sum of inputs for unit j)

o_j = the output computed by unit j

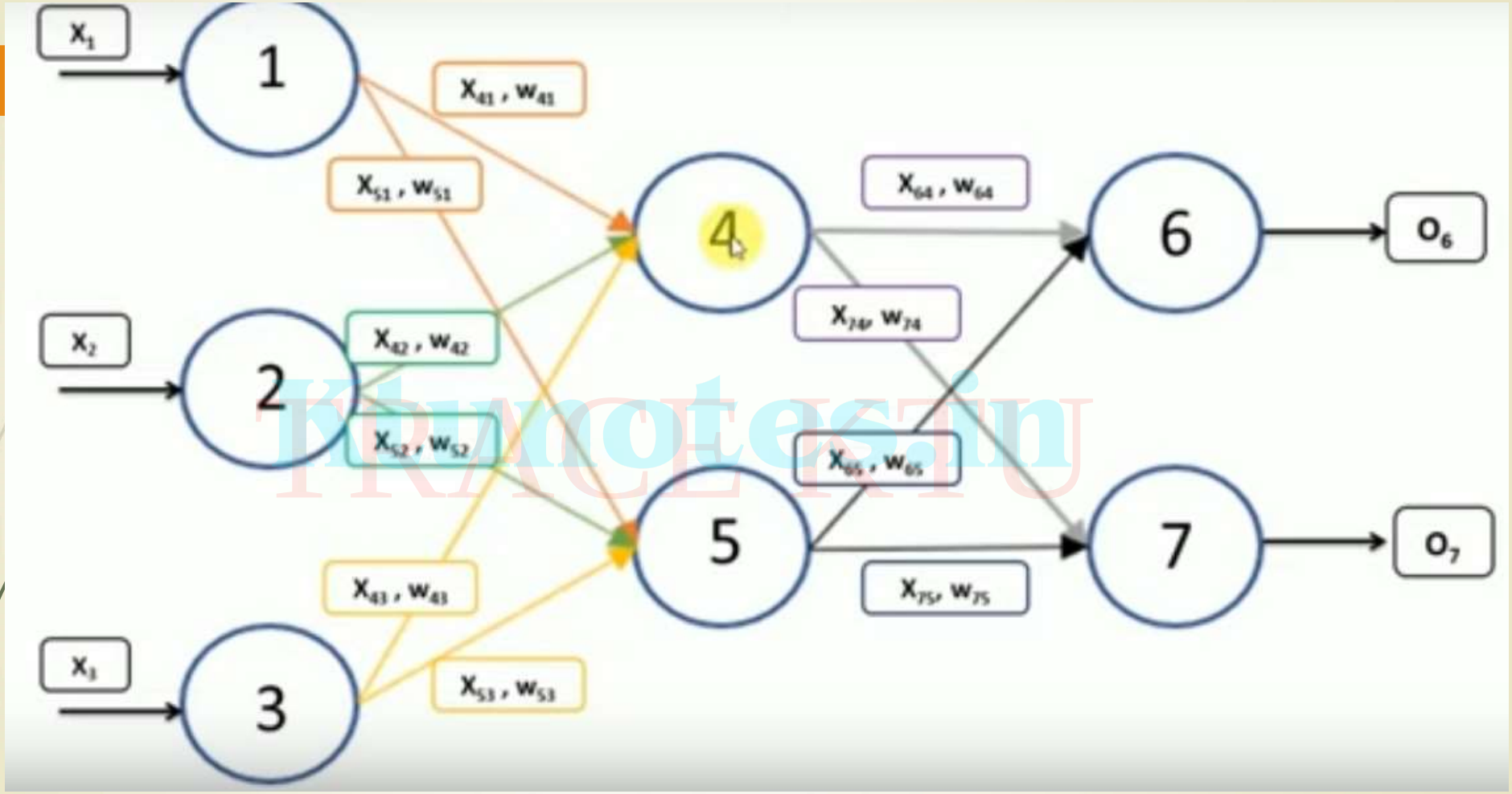
t_j = the target output for unit j

σ = the sigmoid function

outputs = the set of units in the final layer of the network

Downstream(j) = the set of units whose immediate inputs include the output of unit j

KRACLES.in


x_1
1
x_{41}, w_{41}
x_{51}, w_{51}
4
x_{64}, w_{64}
6
o_6
x_2
2
x_{42}, w_{42}
x_{74}, w_{74}
x_{52}, w_{52}
x_{65}, w_{65}
5
7
o_7
x_{43}, w_{43}
x_{75}, w_{75}
x_3
3
x_{53}, w_{53}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

- To begin, notice that weight ***wji*** can influence the rest of the network only through ***netj.***

Therefore, we can use the chain rule to write,

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}}$$

$$net_j = \sum_i w_{ji} x_{ji}$$

$$\frac{\partial net_j}{\partial w_{ji}} = x_{ji}$$

$$= \frac{\partial E_d}{\partial net_j} x_{ji}$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} x_{ji}$$

- Our remaining task is to derive a convenient expression for $\frac{\partial E_d}{\partial net_j}$

Krucial.in

To derive a convenient expression for $\frac{\partial E_d}{\partial net_j}$

We consider two cases in turn:

KRACLES.in

- Case 1, where unit j is an output unit for the network, and
- Case 2, where unit j is an internal unit of the network.

Case 1: Training Rule for Output Unit Weights

- Just as ***wji*** can influence the rest of the network only through ***net$_j$***, ***net$_j$*** can influence the network only through ***oj***. Therefore, we can invoke the chain rule again to write,

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j}\frac{\partial o_j}{\partial net_j}$$

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j}\frac{1}{2}\sum_{k \in outputs}(t_k - o_k)^2$$

$$\frac{\partial o_j}{\partial (net_j)} = \frac{\partial \sigma(net_j)}{\partial (net_j)}$$

$$\frac{\partial \sigma(x)}{\partial (x)} = \sigma(x)\,(1 - \sigma(x))$$

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j}\frac{1}{2}(t_j - o_j)^2$$

$$= \frac{1}{2}2(t_j - o_j)\frac{\partial (t_j - o_j)}{\partial o_j}$$

$$= -(t_j - o_j)$$

$$= \sigma(net_j)\,(1 - \sigma(net_j))$$

$$= o_j\,(1 - o_j)$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j)\,o_j(1 - o_j)$$

Case 1: Training Rule for Output Unit Weights

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j)\, o_j(1 - o_j)$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} x_{ji}$$

$$\Delta w_{ji} = \eta\, (t_j - o_j)\, o_j(1 - o_j) x_{ji}$$

$$\delta_j = (t_j - o_j)\, o_j(1 - o_j)$$

$$\Delta w_{ji} = \eta\, \delta_j\, x_{ji}$$

Kvarotes.in

Case 2: Training Rule for Hidden Unit Weights

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in Downstream(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \, w_{kj} \frac{\partial o_j}{\partial net_j}$$

$$= \sum_{k \in Downstream(j)} -\delta_k \, w_{kj} \, o_j(1 - o_j)$$

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j) \, o_j(1 - o_j)$$

$$\delta_j = (t_j - o_j) \, o_j(1 - o_j)$$

$$\frac{\partial net_k}{\partial o_j} = \frac{\partial x_{kj} w_{kj}}{\partial o_j} = \frac{\partial o_j w_{kj}}{\partial o_j}$$

$$\frac{\partial o_j}{\partial (net_j)} = \frac{\partial \sigma(net_j)}{\partial (net_j)}$$

$$= \sigma(net_j)(1 - \sigma(net_j))$$

$$= o_j(1 - o_j)$$

Kraclesin

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial net_j} x_{ji} \qquad \frac{\partial E_d}{\partial net_j} = \sum_{k \in Downstream(j)} -\delta_k \, w_{kj} \, o_j(1 - o_j)$$

$$\Delta w_{ji} = \eta \, o_j(1 - o_j) \sum_{k \in Downstream(j)} \delta_k \, w_{kj} \, x_{ji}$$

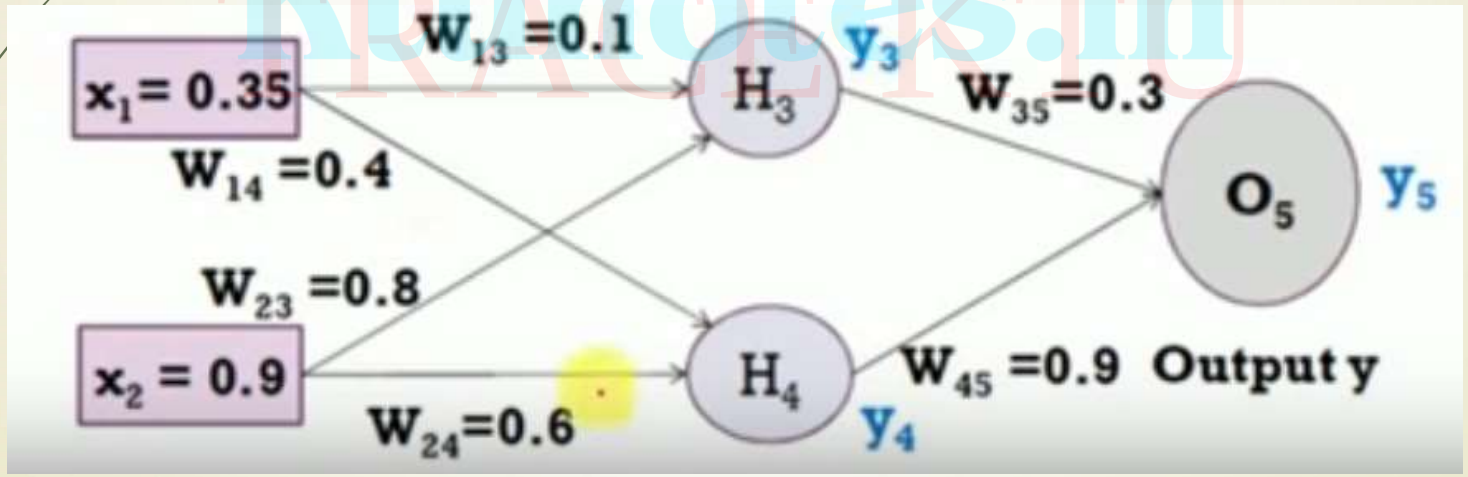
$$\Delta w_{ji} = \eta \, \delta_j \, x_{ji} \qquad \delta_j = o_j(1 - o_j) \sum_{k \in Downstream(j)} \delta_k \, w_{kj}$$

Krucotes.ru
KRUCOTES.RU

Problem 1

Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 0.5 and learning rate is 1. Perform another forward pass.

Kraotes.in



- Forward Pass: Compute output for y3, y4 and y5.

$$a_j = \sum_i (w_{i,j} * x_i) \qquad yj = F(aj) = \frac{1}{1 + e^{-a_j}}$$

$a_1 = (w_{13} * x_1) + (w_{23} * x_2)$
$= (0.1 * 0.35) + (0.8 * 0.9) = 0.755$
$y_3 = f(a_1) = 1/(1 + e^{-0.755}) = 0.68$

$a_2 = (w_{14} * x_1) + (w_{24} * x_2)$
$= (0.4 * 0.35) + (0.6 * 0.9) = 0.68$
$y_4 = f(a_2) = 1/(1 + e^{-0.68}) = 0.6637$

$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$
$= (0.3 * 0.68) + (0.9 * 0.6637) = 0.801$

$y_5 = f(a_3) = 1/(1 + e^{-0.801}) = 0.69$ **(Network output)**

$\text{Error} = y_{target} - y_5 = -0.19$

- Each weight changed by:

$$\Delta w_{ji} = \eta \delta_j o_i$$

$$\delta_j = o_j(1-o_j)(t_j - o_j) \qquad \text{if } j \text{ is an output unit}$$

$$\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj} \qquad \text{if } j \text{ is a hidden unit}$$

Knoldes.in

- where η is a constant called the learning rate
- tj is the correct teacher output for unit j
- δj is the error measure for unit j

- Backward Pass: Compute δ3, δ4 and δ5.

For output unit:

$\delta_5 = y(1-y)\,(y_{target} - y)$

$= 0.69*(1-0.69)*(0.5-0.69) = -0.0406$

KTUACTES.in

For hidden unit:

$\delta_3 = y_3(1-y_3)\, w_{35} * \delta_5$

$= 0.68*(1 - 0.68)*(0.3 * -0.0406) = -0.00265$

$\delta_4 = y_4(1-y_4) w_{45} * \delta_5$

$= 0.6637*(1 - 0.6637)* (0.9 * -0.0406) = -0.0082$

Compute new weights

$$\Delta w_{ji} = \eta \delta_j o_i$$

$\Delta w_{45} = \eta\ \delta_5\ y_4 = 1 * -0.0406 * 0.6637 = -0.0269$

w_{45} (new) $= \Delta w_{45} + w_{45}$(old) $= -0.0269 + (0.9) = 0.8731$

$\Delta w_{14} = \eta\ \delta_4\ x_1 = 1 * -0.0082 * 0.35 = -0.00287$

w_{14} (new) $= \Delta w_{14} + w_{14}$(old) $= -0.00287 + 0.4 = 0.3971$

Knotesin

- Similarly, update all other weights

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
1	3	0.1	-0.00265	0.35	1	0.0991
2	3	0.8	-0.00265	0.9	1	0.7976
1	4	0.4	-0.0082	0.35	1	0.3971
2	4	0.6	-0.0082	0.9	1	0.5926
3	5	0.3	-0.0406	0.68	1	0.2724
4	5	0.9	-0.0406	0.6637	1	0.8731

KRAnotes.in

- Forward Pass: Compute output for y3, y4 and y5.

$$a_j = \sum_i (w_{i,j} * x_i) \qquad yj = F(aj) = \frac{1}{1 + e^{-a_j}}$$

$a_1 = (w_{13} * x_1) + (w_{23} * x_2)$
$= (0.0991 * 0.35) + (0.7976 * 0.9) = 0.7525$
$y_3 = f(a_1) = 1/ (1 + e^{-0.7525}) = 0.6797$

$a_2 = (w_{14} * x_1) + (w_{24} * x_2)$
$= (0.3971 * 0.35) + (0.5926 * 0.9) = 0.6723$
$y_4 = f(a_2) = 1/ (1 + e^{-0.6723}) = 0.6620$

$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$
$= (0.2724 * 0.6797) + (0.8731 * 0.6620) = 0.7631$
$y_5 = f(a_3) = 1/ (1 + e^{-0.7631}) = 0.6820$

Error $= y_{target} - y_5 = -0.182$

KRUCES.in

- **Algorithm: Backpropagation.** Neural network learning for classification or numeric prediction, using the backpropagation algorithm.
- **Input:**
  - *D*, a data set consisting of the training tuples and their associated target values;
  - *l*, the learning rate;
  - *network*, a multilayer feed-forward network.
- **Output:** A trained neural network.
- **Method:**

(1) Initialize all weights and biases in *network*;
(2) **while** terminating condition is not satisfied {
(3) **for** each training tuple X in D {
(4) // Propagate the inputs forward:
(5) **for** each input layer unit j {
(6) $O_j = I_j$; // output of an input unit is its actual input value
(7) **for** each hidden or output layer unit j {
(8) $I_j = \sum_i w_{ij} O_i + \theta_j$; //compute the net input of unit j with respect to the previous layer, i
(9) $O_j = \frac{1}{1+e^{-I_j}}$; } // compute the output of each unit j
(10) // Backpropagate the errors:
(11) **for** each unit j in the output layer
(12) $Err_j = O_j(1 - O_j)(T_j - O_j)$; // compute the error
(13) **for** each unit j in the hidden layers, from the last to the first hidden layer
(14) $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$; // compute the error with respect to the next higher layer, k
(15) **for** each weight w_{ij} in *network* {
(16) $\Delta w_{ij} = (l) Err_j O_i$; // weight increment
(17) $w_{ij} = w_{ij} + \Delta w_{ij}$; } // weight update
(18) **for** each bias θ_j in *network* {
(19) $\Delta \theta_j = (l) Err_j$; // bias increment
(20) $\theta_j = \theta_j + \Delta \theta_j$; } // bias update
(21) } }

KTUNOTES.IN

Problem 2

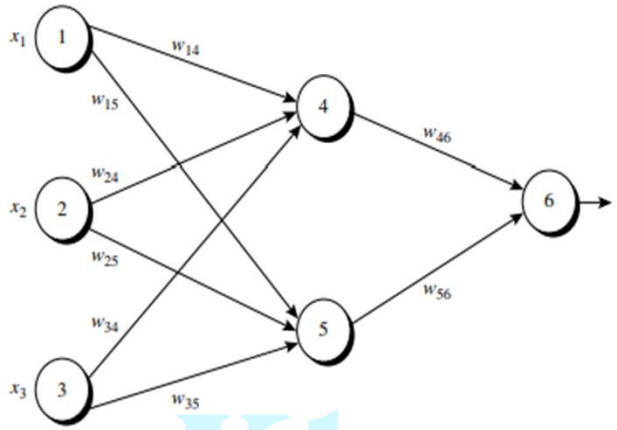


Figure 9.5 Example of a multilayer feed-forward neural network.

Table 9.1 Initial Input, Weight, and Bias Values

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	−0.3	0.4	0.1	−0.5	0.2	−0.3	−0.2	−0.4	0.2	0.1

Table 9.2 Net Input and Output Calculations

Unit, j	Net Input, I_j	Output, O_j
4	$0.2+0-0.5-0.4=-0.7$	$1/(1+e^{0.7})=0.332$
5	$-0.3+0+0.2+0.2=0.1$	$1/(1+e^{-0.1})=0.525$
6	$(-0.3)(0.332)-(0.2)(0.525)+0.1=-0.105$	$1/(1+e^{0.105})=0.474$

Table 9.3 Calculation of the Error at Each Node

Unit, j	Err_j
6	$(0.474)(1-0.474)(1-0.474)=0.1311$
5	$(0.525)(1-0.525)(0.1311)(-0.2)=-0.0065$
4	$(0.332)(1-0.332)(0.1311)(-0.3)=-0.0087$

Table 9.4 Calculations for Weight and Bias Updating

Weight or Bias	New Value
w_{46}	$-0.3+(0.9)(0.1311)(0.332)=-0.261$
w_{56}	$-0.2+(0.9)(0.1311)(0.525)=-0.138$
w_{14}	$0.2+(0.9)(-0.0087)(1)=0.192$
w_{15}	$-0.3+(0.9)(-0.0065)(1)=-0.306$
w_{24}	$0.4+(0.9)(-0.0087)(0)=0.4$
w_{25}	$0.1+(0.9)(-0.0065)(0)=0.1$
w_{34}	$-0.5+(0.9)(-0.0087)(1)=-0.508$
w_{35}	$0.2+(0.9)(-0.0065)(1)=0.194$
θ_6	$0.1+(0.9)(0.1311)=0.218$
θ_5	$0.2+(0.9)(-0.0065)=0.194$
θ_4	$-0.4+(0.9)(-0.0087)=-0.408$

Kuanotes.id

PRACTICAL ISSUES IN NEURAL NETWORK TRAINING

- challenges are primarily related to several practical problems associated with training, the most important one of which is *overfitting*
- ***The Problem of Overfitting and underfitting***
  - there is always a gap between the training and test data performance,
  - In general, models with a larger number of parameters are said to have high capacity, and they require a larger amount of data in order to gain generalization power to unseen test data.

KRACE KTU

How to Detect Overfitting

- The error on test data might be caused by several reasons.

– Other reasons might be bias (underfitting), noise, and poor convergence.

- Overfitting shows up as a large gap between in-sample and out-of-sample accuracy.
- First solution is to collect more data.

– More data might not always be available!

- **High overfitting shows overfitting of data**
- **Data is not cleaned**
- **Model is not trained for sufficient time**

Kvanotes.in

- **High bias in data---Underfitting**
- **Model assumed to be too simple**

Krnotes.in

Regularization

- Since a larger number of parameters causes overfitting, a natural approach is to constrain the model to use fewer non-zero parameters.
- Smaller absolute values of the parameters also tend to overfit less. Since it is hard to constrain the values of the parameters, the softer approach of adding the penalty $\lambda||\overline{W}||^p$ to the loss function is used.
- The value of p is typically set to 2, which leads to ***Tikhonov regularization***
- In general, the squared value of each parameter (multiplied with the regularization parameter $\lambda > 0$) is added to the objective function.
- The practical effect of this change is that a quantity proportional to λwi is subtracted from the update of the parameter wi.
- An example of a regularized version of Equation $\overline{W} \Leftarrow \overline{W} + \alpha \sum_{\overline{X} \in S} E(\overline{X})\overline{X}$ for mini-batch S and update step-size $a > 0$ is as follows:

$$\overline{W} \Leftarrow \overline{W}(1 - \alpha\lambda) + \alpha \sum_{\overline{X} \in S} E(\overline{X})\overline{X}$$

$E[X]$ represents the current error $(y - \hat{y})$ between observed and predicted values of training instance X.

- Regularization is particularly important when the amount of available data is limited
- A neat biological interpretation of regularization is that it corresponds to gradual forgetting, as a result of which "less important" (i.e., *noisy*) patterns are removed.
- **Neural Architecture and Parameter Sharing**
- many of the parameters might be shared. For example, a convolutional neural network uses the same set of parameters to learn the characteristics of a local block of the image.
- The recent advancements in the use of neural networks like recurrent neural networks and convolutional neural networks are examples of this phenomena.

Early Stopping

- Another common form of regularization is *early stopping*, in which the gradient descent is ended after only a few iterations.
- One way to decide the stopping point is by holding out a part of the training data, and then testing the error of the model on the held-out set.
- The gradient-descent approach is terminated when the error on the held-out set begins to rise.
- Early stopping essentially reduces the size of the parameter space to a smaller neighborhood within the initial values of the parameters.
- From this point of view, early stopping acts as a regularizer because it effectively restricts the parameter space.

KRACLES.IN

Trading Off Breadth for Depth

- networks with more layers (i.e., greater depth) tend to require far fewer units per layer because the composition functions created by successive layers make the neural network more powerful.
- Increased constraints reduce the capacity of the network, which is helpful when there are limitations

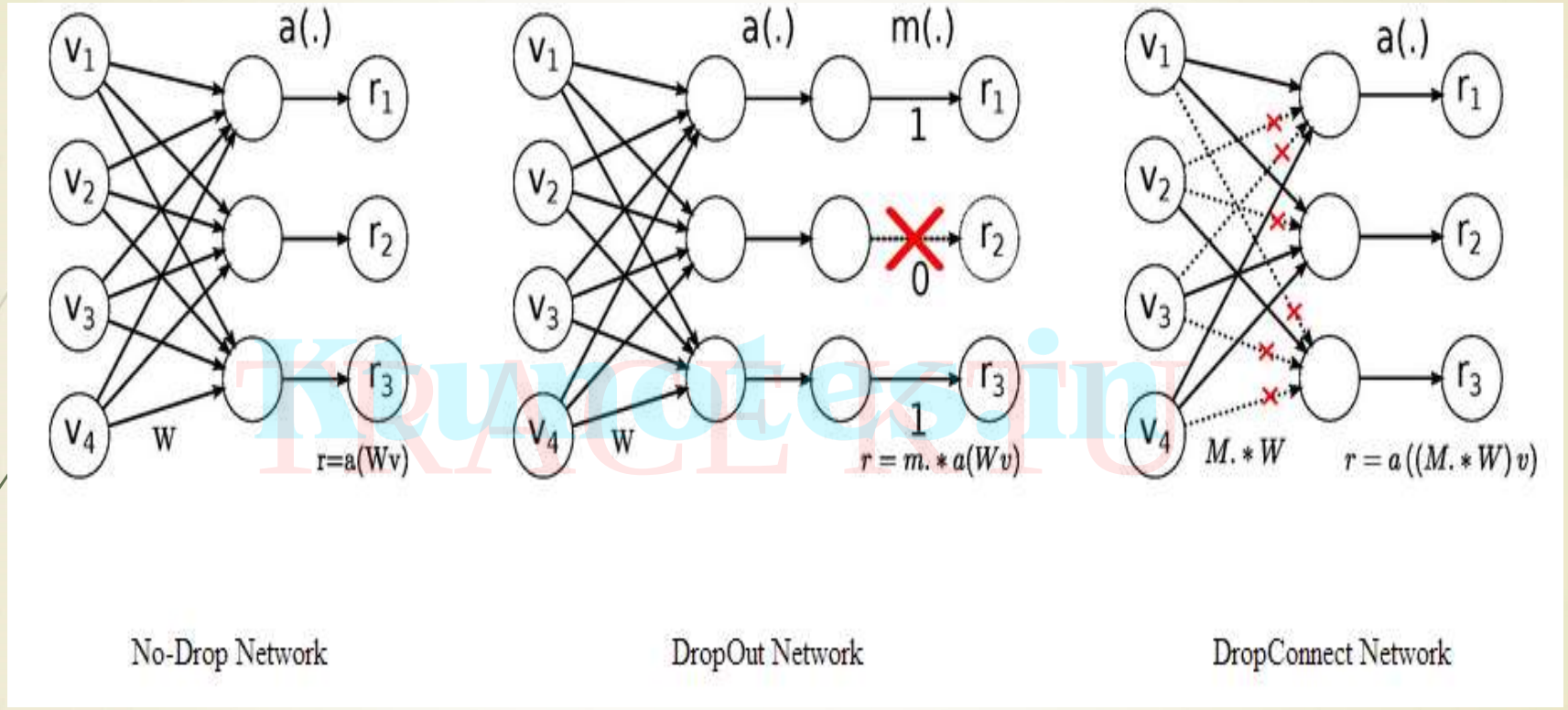
on the amount of available data.

- Even though deep networks have fewer problems with respect to overfitting, they come with a different family of problems associated with ease of training.

Ensemble Methods

- bagging are used in order to increase the generalization power of the model.
- number of ensemble methods that are specifically focused on neural networks have also been proposed. Two such methods include ***Dropout* and *Dropconnect*.**
- When training with Dropout, a randomly selected subset of activations are set to zero within each layer.
- DropConnect instead sets a randomly selected sub-set of weights within the network to zero. Each unit thus receives input from a random subset of units in the previous layer.

KRACKER.in


a(.)
v1
v2
v3
v4
r1
r2
r3
W
r=a(Wv)
No-Drop Network
a(.)
m(.)
1
0
1
r = m. * a(Wv)
DropOut Network
a(.)
M. * W
r = a((M. * W) v)
DropConnect Network

THE VANISHING AND EXPLODING GRADIENT PROBLEMS

- While increasing **depth often reduces the number of parameters of the network**, it leads to different types of practical issues.
- Propagating backwards using the chain rule has its drawbacks in networks with a large number of layers in terms of the stability of the updates.
- the **updates in earlier layers can either be negligibly small (vanishing gradient)** or they can be increasingly large (exploding gradient) in certain types of neural network architectures.
- caused by the chain-like product computation ,which can either exponentially increase or decay over the length of the path.

KRACLEK.IO

- consider a situation in which we have a multi-layer network with one neuron in each layer.
- **Each local derivative along a path can be shown to be the product of the weight and the derivative of the activation function.** The overall backpropagated derivative is the product of these values.
- If each such value is randomly distributed, and has an expected value less than 1, the product of these derivatives will drop off exponentially fast with path length.
- the vanishing and exploding gradient problems are rather natural to deep networks, which makes their training process unstable

TRACE4R.in

- For example, a sigmoid activation often encourages the vanishing gradient problem, because its derivative is less than 0.25 at all values of its argument), and is extremely small at saturation
- **A ReLU activation unit is known to** be less likely to create a vanishing gradient problem because its derivative is always 1 for positive values of the argument
- the use of adaptive learning rates and conjugate gradient methods can help in many cases.
- a recent technique called **batch normalization** is helpful in addressing some of these issues
- Batch Norm is **a normalization technique done between the layers of a Neural Network instead of in the raw data**. It is done along mini-batches instead of the full data set. It serves to speed up training

KNACKER.IO

- In a network of n hidden layers, n derivatives will be multiplied together. If the derivatives are large then the gradient will increase exponentially as we propagate down the model until they eventually explode, and this is what we call the problem of ***exploding gradient***.

KRnotes.in

DIFFICULTIES IN CONVERGENCE

- Sufficiently fast convergence of the optimization process is difficult to achieve with very deep networks, as depth leads to increased resistance to the training process in terms of letting the gradients smoothly flow through the network

LOCAL AND SPURIOUS OPTIMA

- The optimization function of a neural network is highly nonlinear, which has lots of local optima.
- When the parameter space is large, and there are many local optima, it makes sense to spend some effort in picking good initialization points.
- One such method for improving neural network initialization is referred to as pretraining

TRACETK.in

- The basic idea is to use either supervised or unsupervised training on shallow sub-networks of the original network **in order to create the initial weights**
- This type of **pretraining is done in a greedy** and layer wise fashion in which a single layer of the network is trained at one time in order to learn the initialization points of that layer
- The basic idea here is that some of the **minima in the loss function are spurious optima because they are exhibited only in the training data and not in the test data.**
- **unsupervised pretraining often tends to avoid problems associated with overfitting**
- Using unsupervised pretraining tends to move the initialization point closer to the basin of "good" optima in the test data. This is an issue associated with model generalization.

- the notion of spurious optima is often viewed from the lens of model generalization in neural networks
- In traditional optimization, one does not focus on the differences in the loss functions of the training and test data, but on the shape of the loss function in only the training data.
- the problem of local optima (from a traditional perspective) is a smaller issue in neural networks than one might normally expect from such a nonlinear function

COMPUTATIONAL CHALLENGES

- A significant challenge in neural network design is the running time required to train the network
- In recent years, advances in hardware technology such as Graphics Processor Units (GPUs) have helped to a significant extent. GPUs are specialized hardware processors that can significantly speed up the kinds of operations commonly used in neural networks.

- In this sense, some algorithmic frameworks like *Torch* are particularly convenient because they have GPU support tightly integrated into the platform
- One convenient property of the neural network models is that most of the computational heavy lifting ***is front loaded*** during the training phase.
- **the prediction phase** is often computationally efficient, because it requires a small number of operations (depending on the number of layers).
- This is important **because the prediction phase** is often far more time-critical compared to the training phase.

KRACE.in

APPLICATIONS OF NEURAL NETWORKS

- key sectors including finance, healthcare, and automotive They can be used for image recognition, character recognition and stock market predictions,etc.
- **1. Facial Recognition**
- Facial Recognition Systems are serving as robust systems of surveillance. Recognition Systems matches the human face and compares it with the digital images.
- They are used in offices for selective entries. The systems thus authenticate a human face and match it up with the list of IDs that are present in its database.
- **Convolutional Neural Networks (CNN)** are used for **facial recognition and image processing.**

- **2. <u>Stock Market Prediction</u>**
- *Investments are subject to market risks.*
- To make a successful stock prediction in real time a **Multilayer Perceptron MLP** *(class of feedforward artificial intelligence algorithm)* is employed.
- MLP comprises multiple layers of nodes, each of these layers is fully connected to the succeeding nodes. Stock's past performances, annual returns, and non profit ratios are considered for building the MLP model.
- **3. <u>Social Media</u>**
- Artificial Neural Networks are used to study the behaviours of social media users. Data shared everyday via virtual conversations is tacked up and analyzed for competitive analysis.

Kreatek.in

- Neural networks duplicate the behaviours of social media users. Post analysis of individuals' behaviours via social media networks the data can be linked to people's spending habits. **Multilayer Perceptron ANN** is used to mine data from social media application
- **4. Aerospace**
- Aerospace Engineering is an expansive term that covers developments in spacecraft and aircraft.
- Fault diagnosis, high performance auto piloting, securing the aircraft control systems, and modeling key dynamic simulations are some of the key areas that neural networks have taken over. Time delay Neural networks can be employed for modelling non linear time dynamic systems.

KTUNOTES.IN

- **Time Delay Neural Networks** are used for **position independent feature recognition.** The algorithm thus built based on time delay neural networks can recognize patterns.
- **5. <u>Defence</u>**
- Neural Networks also shape the defence operations of technologically advanced countries.
- Neural networks are used in logistics, armed attack analysis, and for object location. They are also used in air patrols, maritime patrol, and for controlling automated drones.
- **Convolutional Neural Networks(CNN),** are employed for determining the presence of underwater mines. Underwater mines are the underpass that serve as an illegal commute route between two countries

KnowledgeIQ

- **6. Healthcare**
- Modern day individuals are leveraging the advantages of technology in the healthcare sector. **Convolutional Neural Networks** are actively employed in the healthcare industry for **X ray detection, CT Scan** and **ultrasound.**
- As CNN is used in image processing, the medical imaging data retrieved from aforementioned tests is analyzed and assessed based on neural network models. **Recurrent Neural Network (RNN)** is also being employed for the development of voice recognition systems.
- **7. Signature Verification and Handwriting Analysis**
- Signature Verification , as the self explanatory term goes, is used for verifying an individual's signature. Banks, and other financial institutions use signature verification to cross check the identity of an individual.

Ktunotes.in

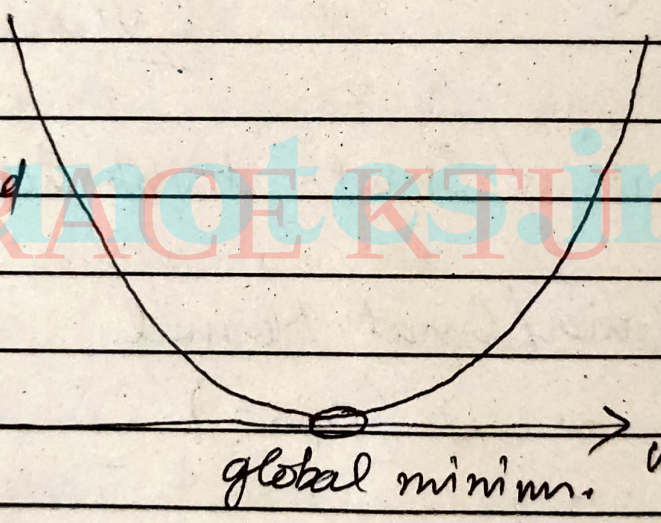
- **Artificial Neural Networks** are used for **verifying the signatures.** ANN are trained to recognize the difference between real and forged signatures. ANNs can be used for the verification of both offline and online signatures.
- **8. <u>Weather Forecasting</u>**
- Forecasting is primarily undertaken to anticipate the upcoming weather conditions beforehand. In the modern era, weather forecasts are even used to predict the possibilities of natural disasters.
- **Multilayer Perceptron (MLP), Convolutional Neural Network (CNN) and Recurrent Neural Networks (RNN)** are used for weather forecasting. Traditional ANN multilayer models can also be used to predict climatic conditions 15 days in advance. A combination of different types of neural network architecture can be used to predict air temperatures.

----------END OF MODULE 1----------

ktunotes.in

GLOBAL Vs LOCAL MINIMA.

During the learning process of neural network we always try to reduce the Error or Loss, by changing the weight. The process can be depicted as shown in the following graph.

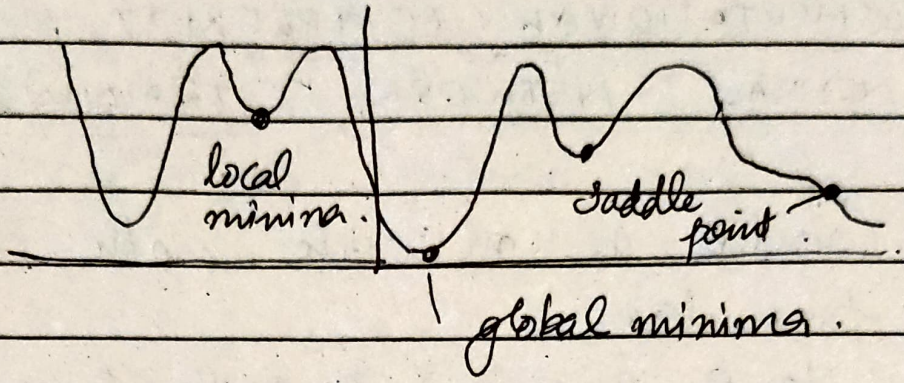


These types of functions are called convex function.

By increasing or decreasing weight we reach global minimum. The weights given at this point will be the best possible weight for that model.

But the curve will not be always like this. For example as shown.

These are non convex functions.



The curve will be having local and global maxima and minima. The derivative at all maxima and minima will be zero. That means slop is zero. Sine derivative is zero, weight update will also be zero. This happens when we are in local minimum also and will not proceed further. So there is always a chance that the learning will converge seeing a saddle point.

COMPUTATIONAL COMPLEXITY IN NEURAL NETWORK TRAINING.

Consider a network with following parameters.

No: of neurons in each layer = n.

No: of layers in network = n.

The complexity in neural network training is associated with two operations.

1) Forward Propogation.
2) Backward Propogation.

Consider Forward Propogation.

It involves computation of output values from each neurons. It is computed as

$$z_i = \sum w_{ij} a_j$$

If there are n input neurons then it has n multiplications which is $O(n)$. There are $n \times n$ neurons in the network and hence.

Complexity = $n^2 O(n) = O(n^3)$

During the Backward propagation the network is calculating the weight updates Δw_{ji} at each neuron inputs. We know that the weight updation

$$\Delta w_{ji} = \eta \delta_j O_i$$

where η is the learning rate.

$\delta_j = O_j(1-O_j)(t_j - O_j)$ for output neuron.

$\delta_j = O_j(1-O_j) \sum_k \delta_k w_{kj}$ for hidden neuron.

and

O_i = is the outputs from previous neuron.

KTRACE.KTU.in

For hidden neuron.

$$\Delta w_{ji} = \eta \delta_j O_i$$

$$= \eta O_j(1-O_j)(t_j - O_j)$$

$$= \eta \left[O_j(1-O_j) \sum \delta_k w_{kj} \right] O_i$$

Here $\sum \delta_k w_{kj}$ denotes sum of product of δ_k and previous layer weight of all input neurons.

If there are n input neurons then.

$\sum_k w_{kj} O_k$ is of complexity $O(n)$.

All other factors form a single computation and the complexity of $\eta O_j(1-O_j) \sum_k w_{kj} O_i$ is $O(n)$.

There are altogether n^2 neurons and of which $2n$ are output/input neurons. So this computation is to performed on (n^2-2n) neurons.

So complexity is $(n^2-2n)\,O(n)$

$= O(n^3)$

To a single neuron there are are a maximum of n ~~weights~~ inputs and hence we need to calculate this for n input weights so

Complexity $= n \times O(n^3)$

$= O(n^4)$

For the output neuron

$$\Delta w_{ji} = \eta \delta_j O_i$$
$$= \eta O_j (1 - O_j)(t_j - O_j).$$
$$= O(1).$$

This computation is done for n output neurons and for each neuron there will be n weight updation to be done which gives the complexity as.

$$= n \times n \times O(1)$$
$$= O(n^2).$$

KTRACKS.in

So the total complexity of back propagation is

$$O(n^4) + O(n^2)$$
$$= \underline{O(n^4)}.$$